# Time Memory Trade-off For Enumeration

Yuanmi Chen[1], Zhao Chen[2], Tingting Guo[3], Chao Sun[4], Weiqiang Wen[5], and Yu Yu[1,6]

[1] East China Normal University
[2] Research Center for Data Hub and Security, Zhejiang Lab, Hangzhou, China
[3] School of Cyber Science and Engineering, Ningbo University of Technology
[4] Southeast University
[5] LTCI, Telecom Paris, Institut Polytechnique de Paris, Paris, France
[6] Shanghai Jiao Tong University

**Abstract.** We incorporate a meet-in-the-middle strategy into the enumeration algorithm, enabling a tunable time-memory trade-off. The algorithm can achieve a minimum asymptotic time complexity of $n^{n/(4e)+o(n)}$, which, in return, demands memory of the same order. This represents a square-root improvement over the state-of-the-art enumeration algorithms. More generally, our approach attains a time complexity of $n^{cn/(2e)+o(n)}$ with memory usage $n^{(1-c)n/(2e)+o(n)}$, where $c$ is any constant satisfying $\frac{1}{2} \leq c < 1$.

Our approach decouples the head and tail blocks of the lattice basis. For a properly selected parameter, each enumeration space becomes asymptotically the square root of the original search space. Each tail vector is then extended to the head block space to find its closest vectors using an efficient neighboring search algorithm. Among all pairs of neighboring vectors that we iterate through, the shortest difference vector is then the solution to the Shortest Vector Problem (SVP).

Apart from the exact version of the algorithm which is of theoretical interest, we also propose heuristic strategies to improve the practical efficiency. First, we show the adaptation of our algorithm to pruned enumeration. Then we show that with a particularly chosen backbone lattice (rescaled $\mathbb{Z}^n$), we are able to accelerate the neighboring search process to an extremely efficient degree. Finally, we optimize parameters and give a practical cost estimation to show how much acceleration we could bring using this new algorithm.

**Keywords:** Shortest vector problem · Enumeration · Lattice.

## 1 Introduction

The hardness of the shortest vector problem(SVP) on lattices is the foundation for many lattice based cryptosystems. While lattice-based schemes are believed to be quantum resistant, the security parameters for any practical cryptosystem would rely on the theoretical and practical complexity of the state of the art algorithms for solving SVP. Among all other SVP algorithms, the most important ones are sieving and enumeration.

Enumeration, initially proposed by Kannan [14] as a deterministic SVP algorithm in the 1980s, has been the subject of extensive research. Hanrot and Stehlé [13] proved that its time complexity can be bounded by $n^{n/2e+o(n)}$, while only requiring polynomial memory. In practice, enumeration algorithms can be significantly accelerated using heuristic analysis and pruning [21,22,12].

Sieving algorithms, which run in time and space $2^{O(n)}$ [1,19,18], have been enhanced by locality sensitive hashing (LSH) [3,5,15,6,2]. LSH improves efficiency by grouping vectors into local buckets, reducing the need for global pairwise comparisons. The current state-of-the-art time complexity of $2^{0.292n+o(n)}$ with matching memory cost is achieved by [2], while [5] offers a memory-time trade-off with $2^{0.2075n+o(n)}$ space and $2^{0.298n+o(n)}$ time. Another notable advancement is the dimension for free strategy [10], which extends the solvable dimension by $\Theta(n/\log n)$, improving practical performance effectively.

Although sieving has an advantage in asymptotic complexity compared to enumeration, in practical implementations (such as in fplll and NTL) [24,23] and for relevant dimensions, enumeration maintained a leading edge for a long time. It was not until the emergence of the dimension for free technology that sieving surpassed enumeration in actual experiment.

Nevertheless, the enumeration algorithm still has great research significance. Its most prominent strength is that its space complexity is polynomial. In contrast, as the dimension of lattice problems increases, sieving algorithms quickly hit the memory ceiling of a single machine, which restricts their use in high-dimensional scenarios.

An interesting research question is whether we can leverage the low space complexity of the enumeration algorithm to achieve improvements in time complexity. Answering this question could lead to more efficient implementations of enumeration algorithms, helping us better understand the relationship between space and time complexity in lattice-based computations. This, in turn, may inspire new optimization techniques for solving lattice problems.

*Contribution.* We incorporate a meet-in-the-middle strategy into the enumeration algorithm, enabling a tunable time-memory trade-off. The algorithm can achieve a minimum asymptotic time complexity of $n^{n/(4e)+o(n)}$, which, in return, demands memory of the same order. This represents a square-root improvement over the state-of-the-art enumeration algorithms. More generally, our approach attains a time complexity of $n^{cn/(2e)+o(n)}$ with memory usage $n^{(1-c)n/(2e)+o(n)}$, where $c$ is any constant satisfying $\frac{1}{2} \leq c < 1$.

Our approach decouples the head and tail blocks of the lattice basis. For a properly selected parameter, each enumeration space becomes asymptotically the square root of the original search space. Each tail vector is then extended to the head block space to find its closest vectors using an efficient neighboring search algorithm. Among all pairs of neighboring vectors that we iterate through, the shortest difference vector is then the solution to the Shortest Vector Problem (SVP).

Apart from the exact version of the algorithm which is of theoretical interest, we also propose heuristic strategies to improve the practical efficiency. First, we

show the adaptation of our algorithm to pruned enumeration. Then we show that with a particularly chosen backbone lattice (rescaled $\mathbb{Z}^n$), we are able to accelerate the neighboring search process to an extremely efficient degree.Finally, we optimize parameters and give a practical cost estimation to show how much acceleration we could bring using this new algorithm.

*Outline.* Section 2 establishes the necessary preliminaries. Section 3 details our proposed algorithm. Section 4 contains a rigorous theoretical analysis, including the derivation of its asymptotic complexity. Section 5 addresses practical adaptation, parameter optimization, and cost estimation. Finally, Section 6 concludes with a discussion of the technique, comparisons to related work, and directions for future enhancements.

## 2 Preliminaries

### 2.1 Notations and Basics

All vectors are denoted by bold lower case letters and are to be understood as column-vectors. Matrices are denoted by bold capital letters. A matrix $\mathbf{B} \in \mathbb{R}^{m \times n}$ is written as a collection of column vectors $\mathbf{B} = [\mathbf{b}_1, \ldots, \mathbf{b}_n]$. When $\mathbf{B} \in \mathbb{R}^{m \times n}$ has full rank $n$, the lattice $\mathcal{L}$ generated by the basis $\mathbf{B}$ is denoted by $\mathcal{L}(\mathbf{B}) = \{\mathbf{Bx} | \mathbf{x} \in \mathbb{Z}^n\}$.

*Lattice volume.* The volume of a lattice $\mathcal{L}(\mathbf{B})$ is $\det \mathcal{L} = \sqrt{\det(\mathbf{B}^\top \mathbf{B})}$, which is invariant under basis changes.

*Minimum.* The first minimum of a lattice $\mathcal{L}$, denoted as $\lambda_1(\mathcal{L})$, is the length of its shortest non-zero vector. The Hermite's constant $\gamma_n$ is defined as the supremum of the ratio $\frac{\lambda_1^2(\mathcal{L})}{(\det \mathcal{L})^{2/n}}$ over all $n$-dimensional lattices $\mathcal{L}$. Minkowski's theorem shows that $\gamma_n \leq 4v_n^{-2/n}$ [16, Chapter 2, p.53], where $v_n$ is the volume of the $n$-dimensional unit ball. The asymptotic approximation $v_n^{-1/n} \sim \sqrt{\frac{n}{2\pi e}}$ holds for large $n$.

*Gaussian Heuristic.* Given an $n$-dimensional lattice $\mathcal{L}$, when the volume of a measurable subset $S \subseteq \mathbb{R}^n$ is $\mathrm{Vol}(S)$, the number of points in $S \cap \mathcal{L}$ is approximately $\mathrm{Vol}(S)/\det \mathcal{L}$. In particular, $\lambda_1(\mathcal{L})$ of a random lattice is believed to be approximately the radius of the $n$-dimensional ball of volume $\det \mathcal{L}$, i.e., $\lambda_1(\mathcal{L}) \approx (\det \mathcal{L}/v_n)^{1/n} \approx \sqrt{n/2\pi e} \cdot (\det \mathcal{L})^{1/n}$.

*Gram-Schmidt Orthogonalisation.* $\mathbf{B}^\star = [\mathbf{b}_1^\star, \ldots, \mathbf{b}_n^\star]$ is an $m$-dimensional orthogonal family defined recursively as $\mathbf{b}_i^\star = \mathbf{b}_i - \sum_{j=1}^{i-1} \mu_{i,j} \mathbf{b}_j^\star$ where $\mu_{i,j} = \frac{\langle \mathbf{b}_i, \mathbf{b}_j^\star \rangle}{\|\mathbf{b}_j^\star\|^2}$. Alternatively we can also write as the unique matrix decomposition of the basis $\mathbf{B}^{\mathrm{T}} = \mathbf{M} \cdot \mathbf{D} \cdot \mathbf{O}$, where

$$\mathbf{M} = \begin{pmatrix} 1 & 0 & \ldots & 0 \\ \mu_{2,1} & 1 & \ldots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ \mu_{n,1} & \ldots & \mu_{n,n-1} & 1 \end{pmatrix}, \mathbf{D} = \begin{pmatrix} \|\mathbf{b}_1^\star\| & 0 & \ldots & 0 \\ 0 & \|\mathbf{b}_2^\star\| & \ldots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & \ldots & \ldots & \|\mathbf{b}_n^\star\|, \end{pmatrix}, \mathbf{O} = \begin{pmatrix} \frac{\mathbf{b}_1^{\star\top}}{\|\mathbf{b}_1^\star\|} \\ \vdots \\ \frac{\mathbf{b}_n^{\star\top}}{\|\mathbf{b}_n^\star\|} \end{pmatrix},$$

and $\mathbf{O}$ is an $n \times m$ row-orthonormal matrix.

*Projection and Sublattices.* We denote the orthogonal projection to the span of $(\mathbf{b}_1, \ldots, \mathbf{b}_i)$ by $\pi_i$, and we denote $\mathbf{B}_{[i,j]} := [\pi_{i-1}(\mathbf{b}_i), \ldots, \pi_{i-1}(\mathbf{b}_j)]$, and $\mathcal{L}_{[i,j]}$ the lattice generated by $\mathbf{B}_{[i,j]}$. Its corresponding decomposition $\mathbf{M}_{[i,j]}$ is the truncation of the matrix $\mathbf{M}$ from the $i$-th row and column to the $j$-th row and column.

*Size Reduction.* A basis $\mathbf{B}$ is *size-reduced* if its GSO family satisfies $|\mu_{i,j}| \leq 1/2$ for all $1 \leq j < i \leq n$.

*HKZ Reduction.* A basis $\mathbf{B}$ is said to be *Hermite-Korkine-Zolotarev-reduced* if it is size-reduced and $\|\mathbf{b}_i\| = \lambda_1(\mathcal{L}_{i,n})$ for all $1 \leq i \leq n$.

*Quasi-HKZ Reduction.* A basis $\mathbf{B}$ is quasi-HKZ-reduced if it is size-reduced, $\|\mathbf{b}_2^\star\| \geq \|\mathbf{b}_1^\star\|/2$, and $\pi_1(\mathbf{B})$ is HKZ-reduced.

For a vector $\mathbf{v} \in \mathbb{R}^n$ we denote $\mathbf{v}_{[i]}$ as its $i$-th entry, and $\mathbf{v}_{[i,j]}$ as the truncation of the vector from the $i$-th to the $j$-th position. $\mathbf{u} = (\mathbf{v}|\mathbf{w})$ should be understood as a new vector $\mathbf{u}$ formed by the concatenation of two vectors $\mathbf{v}$ and $\mathbf{w}$.

Throughout this paper, we are going to represent a lattice vector $\mathbf{w}$ in two ways. $\mathbf{u} \in \mathbb{Z}^n$ is the integral coefficients of $\mathbf{w}$ on the basis $\mathbf{B} \in \mathbb{R}^{n \times n}$, and $\mathbf{v} \in \mathbb{R}^n$ is the representation of vector $\mathbf{w}$ on the rotated basis $\mathbf{M} \cdot \mathbf{D}$. These two representations are always computed and stored simultaneously. We use cursive font like $\mathcal{S}$ to denote a set of lattice vectors or a well-indexed structure of the lattice vectors, so that each element of the set is a vector with both representations $\mathcal{S} = \{(\mathbf{u}, \mathbf{v}) | \ldots\}$.

### 2.2 Enumeration

Given a lattice basis $\mathbf{B} = (\mathbf{b}_1, \mathbf{b}_2, \ldots, \mathbf{b}_n)$, letting $r$ be the estimated upper bound of $\lambda_1(\mathcal{L})$ — either $\|\mathbf{b}_1\|$, the Minkowski bound, or the Gaussian Heuristic estimation — the `enumerate` procedure presented in Algorithm 1 outputs a set of vectors $\mathcal{S}$ satisfying for all $\mathbf{v} \in \mathcal{S}$, $\mathbf{v} \in \mathcal{L}$ and $\|\mathbf{v}\| \leq r$.

The `enumerate` procedure can be seen as a depth first search on the tree, where the $i$-th level nodes are $\mathbf{u}_{[n-i+1,n]} = (u_{n-i+1}, \ldots, u_n)$ satisfying the partial length bound in the projected lattice $\mathbf{u}_{[n-i+1,n]} \cdot \pi_{n-i}(\mathbf{B}_{[n-i+1,n]}) \leq r^2$. This is equivalent to the $i$-th inequality of the following equations.

$$\begin{aligned} u_n^2 \cdot \|\mathbf{b}_n^\star\|^2 &\leq r^2 \\ (u_{n-1} + \mu_{n,n-1} u_n)^2 \cdot \|\mathbf{b}_{n-1}^\star\|^2 + u_n^2 \cdot \|\mathbf{b}_n^\star\|^2 &\leq r^2 \\ &\cdots \\ \sum_{j=1}^{n} \left( u_j + \sum_{i=j+1}^{n} \mu_{i,j} u_i \right)^2 \cdot \|\mathbf{b}_j^\star\|^2 &\leq r^2. \end{aligned} \tag{1}$$

To solve SVP, Kannan proposes an algorithm which computes an HKZ-reduced basis. The main idea is to first prepare a quasi-HKZ-reduced basis, then apply the enumeration algorithm. However, the enumeration step remains the dominant factor in the overall time complexity. Hanrot and Stehlé drew the following conclusion on the time complexity of this algorithm in [13].

**Algorithm 1** `enumerate`

**Input:** a lattice basis $\mathbf{B} = (\mathbf{b}_1, \ldots, \mathbf{b}_n)$, search radius $r \in \mathbb{R}$,
**Output:** a set of non-zero vectors $\mathcal{S} = \{\mathbf{v} \in \mathcal{L} \wedge \|\mathbf{v}\| \leq r\}$

Compute all $\mu_{i,j}$'s and $\|\mathbf{b}_i^\star\|^2$'s.
$\mathbf{u} \leftarrow \mathbf{0}; \mathbf{l} \leftarrow \mathbf{0}; \mathcal{S} \leftarrow \emptyset; i \leftarrow 1$
**while** $i \leq n$ **do**
  $l_i \leftarrow (u_i + \sum_{j>i} u_j \mu_{j,i})^2 \|\mathbf{b}_i^\star\|^2$
  **if** $\sum_{j>i} l_j \leq r^2$ **then** ▷ If within $r^2$ bound
    **if** $i = 1$ **then**
      $\mathcal{S} \leftarrow \mathcal{S} \cup \{\mathbf{u} \cdot \mathbf{B}\}$ ▷ One more solution found
    **else**
      $i \leftarrow i - 1$; $u_i \leftarrow \left\lceil -\sum_{j>i}(x_j \mu_{j,i}) - \sqrt{\frac{r^2 - \sum_{j>i} l_j}{\|\mathbf{b}_i^\star\|^2}} \right\rceil$ ▷ Go down the tree
    **end if**
  **else**
    $i \leftarrow i + 1$; $u_i \leftarrow u_i + 1$ ▷ Beyond $r^2$ bound, go up the tree
  **end if**
**end while**
**return** $\mathcal{S}$

**Theorem 1.** *Given as inputs an n-dimensional basis* $\mathbf{B}$ *ofm-dimensional integer vectors whose entries have absolute values* $\leq B$*, one can compute an HKZ-reduced basis of the lattice spanned by* $\mathbf{B}$ *in deterministic time* $\mathcal{P}(\log B, m) \cdot n^{n/2e+o(n)}$.

Here, $\mathcal{P}(n_1, \ldots, n_i)$ denotes $(\prod_{j=1}^{i} n_i)^c$ for some constant $c > 0$, and $B$ is the maximum absolute value of the entries of the input vectors.

## 3 Algorithm

In this section, we present our algorithm and explain its main components in detail. For now, we leave the complexity analysis to the next section.

### 3.1 Approach

The two-stage enumeration algorithm, denoted as `enum2s`, is presented in Algorithm 2. The core idea of the algorithm is to partition the lattice into the head block $\mathbf{B}_{[1,\eta]}$ and the tail block $\mathbf{B}_{[\eta+1,n]}$, and conduct enumeration separately in these two smaller spaces. The vectors produced from the tail block, $\mathcal{S}_t \subset \mathcal{L}(\mathbf{B}_{[\eta+1,n]})$ are then lifted to the whole lattice space in order to match the vectors enumerated in the head block $\mathcal{S}_h \subset \mathcal{L}(\mathbf{B}_{[1,\eta]})$. For each vector $(\mathbf{u}, \mathbf{v}) \in \mathcal{S}_t$, the sub-procedure `find_neighboring` finds all vectors in $\mathcal{S}_h$ that lie within distance $r - \|\mathbf{v}\|$ from the lifted tail vector $\texttt{lift}(\mathbf{u}, \mathbf{v})$. Consequently, we can find the shortest vector of the lattice: it is either the shortest vector in $\mathcal{S}_h$ or the shortest difference vector between a vector in $\mathcal{S}_t$ after lifting and a vector in $\mathcal{S}_h \cup \{\mathbf{0}\}$.

**Algorithm 2** `enum2s`

**Input:** a lattice basis $\mathbf{B} \in \mathbb{Z}^{n \times n}$, an integer $\eta$, the radius $r$.
**Output:** vector set $\mathcal{S} = \{(\mathbf{u}, \mathbf{v}) | \|\mathbf{v}\| < r\}$

$\mathcal{S}_h \leftarrow$ `enumerate_head`$(\mathbf{B}_{[1,\eta]}, r) \cup \{\mathbf{0}\}$
$\mathcal{S}_t \leftarrow$ `enumerate`$(\mathbf{B}_{\eta+1,n}, r) \cup \{\mathbf{0}\}$
$\mathcal{S} \leftarrow \emptyset$
**for all** $(\mathbf{u}, \mathbf{v}) \in \mathcal{S}_t$ **do**
    $\mathcal{S}_n \leftarrow$`find_neighboring`$(\mathcal{S}_h,$ `lift`$(\mathbf{u}, \mathbf{v}), r - \|\mathbf{v}\|)$
    **for all** $(\tilde{\mathbf{u}}, \tilde{\mathbf{v}}) \in \mathcal{S}_n$ and $\tilde{\mathbf{v}} \neq \mathbf{v}$ **do**
        $\mathcal{S} \leftarrow \mathcal{S} \cup \{(\mathbf{u} - \tilde{\mathbf{u}}, \mathbf{v} - \tilde{\mathbf{v}})\}$
    **end for**
**end for**
**return** $\mathcal{S}$

Note that the enumeration procedure for the head block `enumerate_head` has modified boundaries from the original enumeration procedure to ensure the correctness of the algorithm. We will present the algorithm in Subsection 3.2.

For the tail block, we still deploy the normal `enumerate` procedure. The vectors returned by the tail enumeration procedure live in the space spanned by the tail block $\mathbf{B}_{[\eta+1,n]}$, and need to be lifted to the whole space of $\mathbf{B}$ before they can be compared to the vectors in $\mathcal{S}_h$. This `lift` procedure is described in Algorithm 3. It is a standard Babai-lifting process that extends the given vector from the projected lattice to the full lattice.

**Algorithm 3** `lift`

**Input:** a lattice vector $(\mathbf{u}, \mathbf{v}) \in \mathcal{L}(\mathbf{B}_{[k,n]})$ for $k > 1$.
**Output:** a lifted lattice vector $(\mathbf{u}', \mathbf{v}') \in \mathcal{L}(\mathbf{B}_{[1,n]})$.

$(u_1, \ldots, u_{k-1}) \leftarrow \mathbf{0}, (v_1, \ldots, v_{k-1}) \leftarrow \mathbf{u} \cdot \mathbf{M}_{[k,n;1,k-1]} \cdot \mathbf{D}_{[1,k-1;1,k-1]}$
**for** $i \leftarrow k-1$ **downto** 1 **do**
    **if** $|v_i| > \|\mathbf{b}_i^\star\|/2$ **then**
        $u_i \leftarrow -\left\lfloor \frac{v_i}{\|\mathbf{b}_i^\star\|} \right\rceil$
        $(v_1, v_2, \ldots, v_i) \leftarrow (v_1, v_2, \ldots, v_i) + u_i \cdot \mathbf{M}_{[i,i;1,i]} \cdot \mathbf{D}_{[1,i;1,i]}$
    **end if**
**end for**
**return** $(\mathbf{u}' = ((u_1, \ldots, u_{k-1})|\mathbf{u}), \mathbf{v}' = ((v_1, \ldots, v_{k-1})|\mathbf{v}))$

### 3.2 Head Block Enumeration

The head enumeration procedure needs a different boundary to ensure the correctness of the algorithm.The enumeration region must be extended to ensure that $(\mathcal{S}_h - \texttt{lift}(\mathcal{S}_t))$ encompasses $\mathsf{Ball}_{[1,\eta]}(r)$.

Note that vectors in $\mathcal{S}_t$ are first lifted to the full space, then size-reduced.Their first $\eta$ indices, $\mathbf{v}'_{[1,\eta]}$, are distributed within a hyperrectangle with sides $\frac{1}{2}\|\mathbf{b}_i^\star\|$.

That is, for all $1 \leq i \leq \eta$, the following condition holds:

$$-\frac{1}{2} \leq \frac{\mathbf{v}'[i]}{|\mathbf{b}^\star i|} \leq \frac{1}{2}. \tag{2}$$

We call this a *Babai box* (of dimensions $[1, \eta]$) and denote it as $\mathsf{Box}_{[1,\eta]}$, or briefly $\mathsf{Box}$. We use $\mathsf{Ball}_{[i,j]}(r)$ to denote the ball of radius r in $\mathsf{Span}(\mathbf{B}_{[i,j]})$. To guarantee that the vectors iterated in the `enum2s` algorithm include all vectors of norm smaller than r, it is necessary to include the set $\mathsf{Ball}_{[1,\eta]}(r) + \mathsf{Box}_{[1,\eta]}$ in the boundary. A brief two-dimensional illustration of this relationship is given in Figure 5.

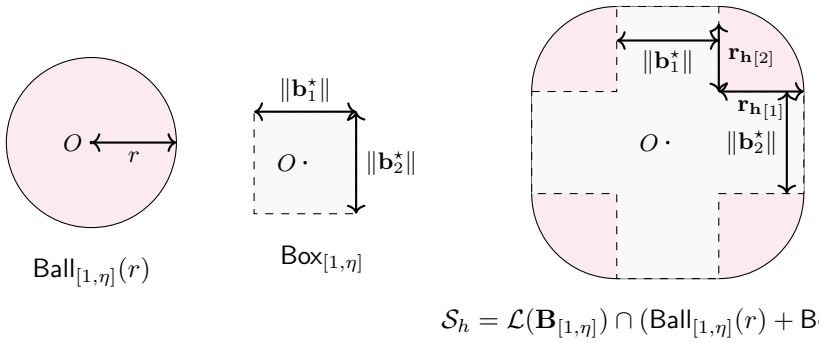



Fig. 1: The enumeration boundary for $\mathcal{S}_h$ to ensure $\mathsf{Ball}(r) \subseteq (\mathcal{S}_h - \mathtt{lift}(\mathcal{S}_t))$, where $\mathsf{Ball}(r)$ is the ball in the full lattice space, and $\mathtt{lift}(\mathcal{S}_t)$ denotes the lifted tail vectors

**Theorem 2 (Head Enumeration Region).** *Let* $\mathcal{S}_h = \mathcal{L}(\mathbf{B}_{[1,\eta]}) \cap (\mathsf{Box}_{[1,\eta]} + \mathsf{Ball}_{[1,\eta]}(r))$ *and* $\mathcal{S}_t = \mathcal{L}(\mathbf{B}_{[\eta+1,n]}) \cap \mathsf{Ball}_{[\eta+1,n]}(r)$. *For any* $\mathbf{v} \in \mathcal{L} \cap \mathsf{Ball}(r)$ *(i.e., any lattice vector with* $\|\mathbf{v}\| < r$*), one can always find* $\mathbf{v}_h \in \mathcal{S}_h \cup \{0\}$ *and* $\mathbf{v}_t \in \mathcal{S}_t \cup \{0\}$ *such that* $\mathbf{v} = \mathbf{v}_h - \mathbf{v}_t$.

*Proof.* Let $\mathbf{v} = \mathbf{u} \cdot \mathbf{B}$ be the integral representation over the basis $\mathbf{B}$. Then $\mathbf{v}_t = \mathbf{u}_{[\eta+1,n]} \cdot \mathbf{B}_{[\eta+1,n]}$ belongs to $\mathcal{S}_t$ because $\|\mathbf{v}_t\| \leq \|\mathbf{v}\| \leq r$ and $\mathbf{v}_t \in \mathcal{L}(\mathbf{B}_{[\eta+1,n]})$. Let $\mathbf{v}_t''$ be the lifted and size-reduced vector of $\mathbf{v}_t$ with respect to $\mathbf{B}_{[1,\eta]}$; then $\mathbf{v}_t'' \cap \mathsf{Span}(\mathbf{B}_{[1,\eta]}) \subset \mathsf{Box}_{[1,\eta]}$ due to the property of size reduction. Furthermore, $\mathbf{v} - \mathbf{v}_t'' \in \mathcal{L}(\mathbf{B}_{[1,\eta]}) \cap (\mathsf{Ball}_{[1,\eta]}(r) + \mathsf{Box}_{[1,\eta]})$ because it is an integral combination of $\mathbf{B}_{[1,\eta]}$, $\mathbf{v}_{[1,\eta]} \subset \mathsf{Ball}_{[1,\eta]}(r)$, and $\mathbf{v}_{t[1,\eta]}'' \subset \mathsf{Box}_{[1,\eta]}$. Consequently, $\mathbf{v} - \mathbf{v}_t'' \in \mathcal{S}_h$. □

This enumeration shape necessitates a modified enumeration procedure, which is described in Algorithm 4. For the sake of simplicity in description, we define the function $\phi$ for $u \in \mathbb{R}$ as follows:

$$\phi(u) = \max\left(|u| - \frac{1}{2}, 0\right). \tag{3}$$

For each layer in the enumeration, we relax an additional $\pm\frac{1}{2}\|\mathbf{b}_i^\star\|$ with respect to the original radius $r$. This amount is not only added to the enumeration range in Line 8, but also subtracted when computing the partial sum of the vector length in Line 7. The rest of the enumeration procedure remains unchanged.

**Algorithm 4** `enumerate_head`

**Input:** a lattice basis $\mathbf{B}$ of dimension $\eta$, radius vector $\mathbf{r} \in \mathbb{R}^\eta$.
**Output:** vector set $\mathcal{S} = \{(\mathbf{u}, \mathbf{v})\}$, where $\mathcal{S} = \mathcal{L}(\mathbf{B}_{[1,\eta]}) \cap (\mathsf{Ball}_{[1,\eta]}(r) + \mathsf{Box}_{[1,\eta]})$

1: **if** $n = 0$ **then return** $\{(\mathbf{0}, \mathbf{0})\}$
2: **end if**
3: $\mathcal{S}' \leftarrow \texttt{enumerate}(\pi_1(\mathbf{B}), \mathbf{r}_{[2,\eta]})$
4: $\mathcal{S} \leftarrow \emptyset$
5: **for all** $(\mathbf{u}, \mathbf{v}) \in \mathcal{S}'$ **do**
6: $\quad y \leftarrow \sum_{i=2}^{\eta} \mu_{i,1}\mathbf{u}_{[i]}$
7: $\quad \psi \leftarrow \mathbf{r}_{[1]}^2 - \sum_{i=2}^{\eta} \phi\left(\mathbf{u}_{[i]} + \sum_{j=i+1}^{\eta} \mu_{j,i}\mathbf{u}_{[j]}\right)^2 \|\mathbf{b}_i^\star\|^2$
8: $\quad$ **for all** $x \in \mathbb{Z}$ and $|x + y| < \frac{\sqrt{\psi}}{\|\mathbf{b}_1\|} + \frac{1}{2}$ **do**
9: $\quad\quad \mathbf{u}' \leftarrow (x, \mathbf{u}), \mathbf{v}' \leftarrow ((x + y) \cdot \|\mathbf{b}_1\|, \mathbf{v})$
10: $\quad\quad \mathcal{S} \leftarrow \mathcal{S} \cup \{(\mathbf{u}', \mathbf{v}')\}$
11: $\quad$ **end for**
12: **end for**
13: **return** $\mathcal{S}$

Here, we have presented the changes to the head enumeration with regard to the full enumeration, and it can also be naturally adapted to pruned enumeration in practice. This adapted pruned head enumeration will be presented in Section 5. We will also show in the analysis part that this modification to the algorithm does not add to the asymptotic complexity of the algorithm. In the next subsection, we shall present our algorithm for finding neighboring vectors in $\mathcal{S}_h$ for each vector in $\mathcal{S}_t$.

### 3.3 Find Neighboring Vectors

*Problem Description.* Given a set of vectors $\mathcal{S}$ which is a subset of some lattice $\mathcal{L}$, a real number r, and a query vector $\mathbf{u}$, the problem asks to find all vectors in $\mathcal{S}$ within distance r to $\mathbf{u}$ with preprocessing.

As we need to search an area for all vectors within a given distance, a natural thought is to split this large area into smaller sections. Then, for each query, we only need to search the sections close to the query point, instead of blindly searching the entire area. We propose a specific structure to divide the space which we call "beehives". More precisely, the beehive (denoted $\mathcal{B}$) is a data

structure comprised of a series of Voronoi cells of a chosen backbone lattice $\mathcal{L}'$ to ensure efficient discovery of relevant cells.

In the preprocessing stage, all the vectors in $\mathcal{S}$ are placed into the corresponding beehive $\mathcal{B}$, and in the query phase, we check all beehives relevant to the query vector $\mathbf{u}$ to find all vectors within distance $r$ from $\mathbf{u}$.

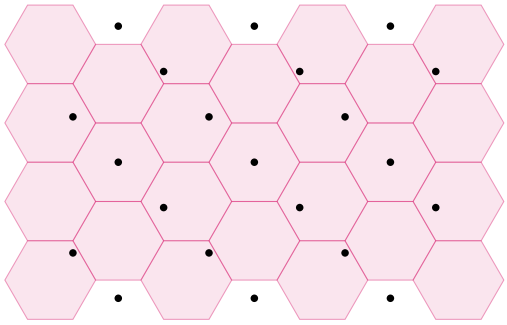


Fig. 2: Pre-processing: place $\mathcal{S} \subset \mathcal{L}$ into the beehive $\mathcal{B}$ with backbone lattice $\mathcal{L}'$

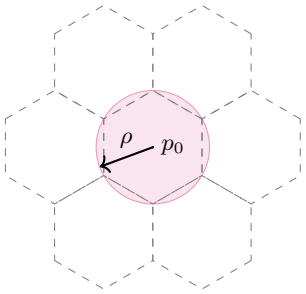



Fig. 3: Covering radius $\rho$ of lattice $\mathcal{L}'$

For clarity, we list the symbols used in Table 1. To distinguish the vector points in $\mathcal{L}'$ and the vectors that we pre-process and query, the former is called *points*, while the latter is called *vectors* throughout this work.

Another important property is the *covering radius*, which is a key property of lattices. It is the smallest non-negative real number $\rho$ such that every point in $\mathbb{R}^n$ lies within distance $\rho$ of at least one lattice point in $\mathcal{L}$.

**Definition 1 (Covering Radius).** *The covering radius of a lattice $\mathcal{L} \subseteq \mathbb{R}^n$ is defined as*

$$\rho = \inf \{r \geq 0 \mid \forall \mathbf{x} \in \mathbb{R}^n, \exists \mathbf{y} \in \mathcal{L} : \|\mathbf{x} - \mathbf{y}\| \leq r\}.$$

In particular, the covering radius of a lattice is the circumradius of its Voronoi cells.

Table 1: Table of Symbols

| Symbol | Description |
|---|---|
| $\mathcal{S}$ | input vector set |
| $\mathcal{L}$ | the lattice containing the input vectors, where $\mathcal{S} \subset \mathcal{L}$ |
| $\mathcal{B}$ | the beehive structure |
| $\mathcal{L}'$ | the backbone lattice of $\mathcal{B}$ |
| $\mathbf{p}$ | $\mathbf{p} \in \mathcal{L}'$ (a point in $\mathcal{L}'$) is the beehive center |
| $\rho$ | the covering radius of $\mathcal{L}'$ |

A beehive structure $\mathcal{B}$ is a collection of Voronoi cells of the backbone lattice $\mathcal{L}'$. Each cell can be naturally represented by its center point $\mathbf{p} \in \mathcal{L}'$. We define the following operations on each beehive represented by $\mathbf{p}$:

- `p.list()`: returns the list of vectors associated with beehive $\mathbf{p}$.
- `p.insert(u)`: inserts a vector $\mathbf{u}$ into the beehive $\mathbf{p}$.

These operations are basic list operations and have constant time complexity. We also define the following operations over all beehives:

- `findAllBeehivesWithin(r,u)`: returns a list of beehives whose center $\mathbf{p}$ are within distance $r$ to $\mathbf{u}$. See Algorithm 5 for details.
- `findClosestBeehive(u)`: returns a single beehive $\mathbf{p}$ which is the closest to $\mathbf{u}$.

Both operations actually reduce to solving variants of the closest vector problem (CVP) on the backbone lattice $\mathcal{L}'$ with target vector $\mathbf{u}$: the former solves a bounded CVP that finds all points within radius $r$, while the latter solves the standard CVP that finds a single closest point. They can be solved via lattice enumeration over the basis of $\mathcal{L}'$. The `findAllBeehivesWithin(r,u)` process, which implements the aforementioned lattice enumeration to solve the CVP, is described in Algorithm 5, where we highlight the differences compared to Algorithm 1. Naturally, the complexity analysis proceeds in an almost identical manner.

**Algorithm 5** `findAllBeehivesWithin`

**Input:** a lattice basis $\mathbf{B} = (\mathbf{b}_1, \ldots, \mathbf{b}_n)$, search radius $r \in \mathbb{R}^+$, and a query vector $(\mathbf{u}, \mathbf{v})$.
**Output:** a set of vectors $\mathcal{S} = \{\mathbf{p} \in \mathcal{L}' \wedge \|\mathbf{p} - \mathbf{u}\| \leq r\}$

Compute $\mu_{i,j}$'s and $\|\mathbf{b}_i^\star\|^2$'s.
$\mathbf{u} \leftarrow \mathbf{0}; \mathbf{l} \leftarrow \mathbf{0}; \mathcal{S} \leftarrow \emptyset; i \leftarrow 1$
**while** $i \leq n$ **do**
  $l_i \leftarrow ((u_i + \sum_{j>i} u_j \mu_{j,i}) \|\mathbf{b}_i^\star\| - v_i)^2$
  **if** $\sum_{j>i} l_j \leq r^2$ **then** ▷ If within $r^2$ bound
    **if** $i = 1$ **then**
      $\mathcal{S} \leftarrow \mathcal{S} \cup \{\mathbf{u} \cdot \mathbf{B}\}$ ▷ One more solution found
    **else**
      $i \leftarrow i - 1$; $u_i \leftarrow \left\lceil v_i - \sum_{j>i} (x_j \mu_{j,i}) - \sqrt{\frac{r^2 - \sum_{j>i} l_j}{\|\mathbf{b}_i^\star\|^2}} \right\rceil$ ▷ Go down the tree
    **end if**
  **else**
    $i \leftarrow i + 1$; $u_i \leftarrow u_i + 1$ ▷ Beyond $r^2$ bound, go up the tree
  **end if**
**end while**
**return** $\mathcal{S}$

**Algorithm 6** `preprocessing`

**Input:** A set of vectors $\mathcal{S} \subset \mathcal{L}$.
**Output:** A beehive structure $\mathcal{B}$ that forms a disjoint partition of $\mathcal{S}$.

```
for all v ∈ S do
    p ← findClosestBeehive(v)
    p.insert(u).
end for
```

In the pre-processing algorithm described in Algorithm 6, we examine each vector $\mathbf{v} \in \mathcal{S}$, and find all beehives whose center $\mathbf{p}$ is closest to $\mathbf{v}$. By the end of preprocessing, the $\mathcal{B}$ should be a disjoint partition of the set $\mathcal{S}$.

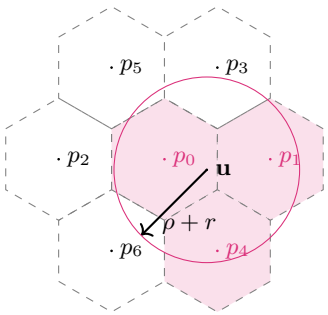



Fig. 4: Query: check all beehives whose center $\mathbf{p}$ lies within distance $\rho + r$ to $\mathbf{u}$

Given a query vector $\mathbf{u}$, it suffices to find all the beehives $\mathbf{p}$ within distance $r + \rho$, and output the vector in `p.list()` within distance $r$ to $\mathbf{u}$. Since $\mathbf{v}$ is at most distance $\rho$ to $\mathbf{p}$, then all the vectors within distance $r$ are guaranteed to be included in `p.list()`. The formal description of the algorithm is in Algorithm 7, and illustrated in Figure 4.

## 4 Complexity Analysis

In this section, we provide a detailed time and memory complexity analysis of `enum2s` (Algorithm 2), where consists of two phases, head enumeration and tail enumeration.

*Time Complexity.* The overall time complexity of `enum2s` is decomposed into the costs of its two phases and the associated query operations, given by:

$$T = T_{\texttt{enum\_head}} + T_{\texttt{enum\_tail}} \times (1 + T_{\texttt{query}}), \tag{4}$$

where:

**Algorithm 7** `query`

**Input:** A query vector $\mathbf{u}$, search radius $r \in \mathbb{R}^+$, and covering radius $\rho$ of $\mathcal{L}'$.
**Output:** A set $\mathcal{S} = \{\mathbf{v} \mid \|\mathbf{v} - \mathbf{u}\| \leq r\}$.

```
S ← ∅
for all p ∈ findAllBeehivesWithin(r + ρ, u) do
    for all v ∈ p.list() do
        if ‖v − u‖ ≤ r then
            S ← {v} ∪ S
        end if
    end for
end for
return S
```

- $T_{\texttt{enum\_head}}$: Time complexity of the head enumeration phase, which generates a candidate set $\mathcal{C}_h$ (size $\#(\mathcal{C}_h)$) by enumerating lattice points in a bounded region.

- $T_{\texttt{enum\_tail}}$: Time complexity per iteration of the tail enumeration phase, which processes each candidate in $\mathcal{C}_h$.

- $T_{\texttt{query}}$: Time complexity of a single `query` operation (Algorithm 7), used to validate candidates in the tail phase. The term $(1 + T_{query})$ accounts for one base iteration step plus one query per tail iteration.

*Space Complexity* The space complexity is determined by the storage overhead of $\mathcal{S}_h$ generated in the head enumeration phase, given by:

$$S = \#(\mathcal{S}_h). \tag{5}$$

In Subsection 4.1, we will show that the modification to `enumeration_head` does not change the asymptotic complexity; neither does it change the multiplicative nature of the overall complexity. That is, for any given $0 < c < 1$, there exists a dividing index $\eta$ that divides the enumeration space of the lattice $\mathcal{L}$ (generated by $\mathbf{B}$) into the head lattice $\mathcal{L}_{[1,\eta]}$ (generated by $\mathbf{B}_{[1,\eta]}$) and the tail lattice $\mathcal{L}_{[\eta+1,n]}$ (generated by $\mathbf{B}_{[\eta+1,n]}$), such that $T_{\texttt{enum\_head}} = n^{cn/(2e)+o(n)}$ and $T_{\texttt{enum\_tail}} = n^{(1-c)n/(2e)+o(n)}$.

We list a summary of complexity results in Table 2, with detailed derivations presented in the next section.

### 4.1 Complexity of Head Enumeration

To study the time complexity $T_{\texttt{enum\_head}}$, we first show that the modified `enumerate_head` process retains the same asymptotic complexity as the original enumeration procedure analyzed in prior work. This consistency is rooted in the analytical framework established by Hanrot and Stehlé [13], which we extend here. We then demonstrate that splitting the enumeration into two stages can reduce the time complexity to the square root level, albeit with increased memory usage.

| Component | Complexity | Best complexity ($c = 1/2$) |
|---|---|---|
| $T_{\texttt{enum\_head}}$ | $n^{\frac{cn}{2e}+o(n)}$ | $n^{\frac{n}{4e}+o(n)}$ |
| $T_{\texttt{enum\_tail}}$ | $n^{\frac{(1-c)n}{2e}+o(n)}$ | $n^{\frac{n}{4e}+o(n)}$ |
| $T_{\texttt{query}}$ | $2^{O(n)}$ | $2^{O(n)}$ |
| Total time complexity ($T$) | $n^{\max(c,1-c)\cdot\frac{n}{2e}+o(n)}$ | $n^{\frac{n}{4e}+o(n)}$ |
| Space complexity ($S$) | $n^{\frac{cn}{2e}+o(n)}$ | $n^{\frac{n}{4e}+o(n)}$ |

Table 2: Summary of Complexity Results for Main Components of `enum2s`

Recall from Section 2 that Hanrot and Stehlé derived a key result for the time complexity of lattice enumeration algorithms (Theorem 1): for an n-dimensional basis $\mathbf{B}$ of integer vectors with entries bounded by B, the deterministic time to compute an HKZ-reduced basis is $P(\log B, m) \cdot n^{n/(2e)+o(n)}$, where $P(\cdot)$ denotes a polynomial in its arguments. Our analysis of `enumerate_head` follows this framework, with the core insight that the modified procedure, despite augmenting the searched volume, does not alter the asymptotic growth rate of the complexity bound, remaining consistent with the original result in [13].

To quantify this bound, we follow the analytical framework in [13]. Up to some polynomial in $n$ and $\log \|\mathbf{B}\|$, the time complexity of `enumerate_head` is asymptotically equivalent to the number of nodes in the enumeration tree, as each node corresponds to a single enumeration step. Specifically, we bound the number of nodes generated in the enumeration procedure of the head enumeration algorithm (Algorithm 4) by counting valid tuples that satisfy the following constraint:

$$\sum_{i=1}^{\eta} \left| \left\{ (x_i, \ldots, x_\eta) \in \mathbb{Z}^{\eta-i+1} | \sum_{j \geq i} \phi \left( x_j + \left( \sum_{k>j} \mu_{k,j} x_k \right) \right)^2 \cdot \|\mathbf{b}_j^\star\|^2 \leq r^2 \right\} \right|, \tag{6}$$

where $\phi(x) = \max(|x| - \frac{1}{2}, 0)$ is an adaptation to the enumeration region of the `enumerate_head` procedure, differing from the original enumeration. In the following, we have highlighted in red all the parts that differ from the original deduction.

To simplify the bound, we define $\psi(x) = \max(|x| - 1, 0)$. For any $\epsilon \in [-1/2, 1/2]$ and $x \in \mathbb{Z}$, $\phi(x+\epsilon)^2 \geq \psi(x)^2$ holds. This inequality allows us to relax the constraint by replacing $\phi$ with the looser bound $\psi$ and derive a more tractable upper bound for the complexity of `enumerate_head`, as follows:

$$\sum_{i=1}^{\eta} \left| \left\{ (x_i, \ldots, x_\eta) \in \mathbb{Z}^{\eta-i+1} \mid \sum_{j \geq i} \psi(x_j)^2 \cdot |\mathbf{b}_j^\star|^2 \leq r^2 \right\} \right|. \tag{7}$$

Following similar deduction path from [13], we define the convex body

$$\mathcal{E}_i = \left\{ (y_i, \ldots, y_\eta) \in \mathbb{Z}^{\eta-i+1}, \sum_{j \geq i} \psi(y_j)^2 \cdot \|\mathbf{b}_j^\star\|^2 \leq 4r^2 \right\},$$

as well as the the quantity $N_i = |\mathcal{E}_i \cap \mathbb{Z}^{d-i+1}|$. Using the relation

$$\sum_{x \in \mathbb{Z}} f(\psi(x)) = \sum_{x \in \mathbb{Z}} f(x) + 2 \cdot f(0),$$

we can bound the value $N_i$ by:

$$\begin{aligned} N_i = \sum_{(x_i,\ldots,x_\eta) \in \mathbb{Z}^{\eta-i+1}} \mathbf{1}_{(x_i,\ldots,x_\eta)\in\mathcal{E}_i} &\leq \exp\left(\eta\left(1 - \sum_{j\geq i} \psi(x_j)^2 \frac{\|\mathbf{b}_j^\star\|^2}{r^2}\right)\right) \\ \leq e^\eta \cdot \prod_{j\geq i} \sum_{x\in\mathbb{Z}} \exp\left(-\psi(x)^2 \frac{\eta\|\mathbf{b}_j^\star\|^2}{r^2}\right) &= e^\eta \cdot \prod_{j\geq i}\left(2 + \sum_{x\in\mathbb{Z}} \exp\left(-x^2 \frac{\eta\|\mathbf{b}_j^\star\|^2}{r^2}\right)\right) \\ &= e^\eta \cdot \prod_{j\geq i} \Theta\left(\frac{\eta\|\mathbf{b}_j^\star\|^2}{r^2}\right) \end{aligned}$$

where $\Theta(t) = 2 + \sum_{x\in\mathbb{Z}} \exp(-tx^2)$ is defined for $t > 0$. Notice that $\Theta(t) = 3 + 2\sum_{x\geq 1} \exp(-tx^2) \leq 3 + 2\int_0^\infty \exp(-tx^2)dx = 3 + \sqrt{\frac{\pi}{t}}$. Hence $\Theta(t) \leq \frac{3+\sqrt{\pi}}{\sqrt{t}}$ for $t \leq 1$ and $\Theta(t) \leq 3 + \sqrt{\pi}$ for $t \geq 1$. As a consequence, we have:

$$N_i \leq (e(3+\sqrt{\pi}))^\eta \cdot \prod_{j\geq i} \max\left(1, \frac{r}{\sqrt{\eta}\|\mathbf{b}_i^\star\|}\right). \tag{8}$$

Thus one can conclude:

$$T_{\texttt{enum\_head}} = \mathcal{P}(n, \log r, \log B) \cdot 2^{O(\eta)} \cdot \max_{I \subseteq [\![1,\eta]\!]} \left(\frac{r^{|I|}}{(\sqrt{\eta})^{|I|}\prod_{i\in I}\|\mathbf{b}_i^\star\|}\right), \tag{9}$$

where $\mathcal{P}$ denotes a polynomial in its arguments, and $[\![a, b]\!]$ denotes the set of integers from $a$ to $b$. We can see that the difference has been absorbed in the single exponential factor and does not affect the asymptotic complexity.

### 4.2 Complexity of $T_{\texttt{enum\_head}}$ and $T_{\texttt{enum\_tail}}$

Another crucial element to establish the complexity bound for the original enumeration is Theorem 3 of [13], which we restate below.

**Theorem 3.** *Let $\mathbf{b}_1, \ldots, \mathbf{b}_n$ be a quasi-HKZ-reduced basis. Let $I \subseteq [\![1, n]\!]$. Then,*

$$\frac{\|\mathbf{b}_1\|^{|I|}}{\prod_{i\in I}\|\mathbf{b}_i^\star\|} \leq (\sqrt{n})^{|I|\left(1+\log\frac{n}{|I|}\right)} \leq (\sqrt{n})^{\frac{n}{e}+|I|}$$

Substitute this expression into Equation 9, and then set $r = \mathbf{b}_1$, we arrive at the classic complexity bound of bound of $n^{n/(2e)+o(n)}$ for the original enumeration.

In the following, we reuse Theorem 3 to prove our complexity bound. For simplicity, we define

$$f(I) = \frac{\|\mathbf{b}_1\|^{|I|}}{\prod_{i\in I}\|\mathbf{b}_i^\star\|}.$$

By restating Theorem 3 using this definition, we obtain the key inequality for $f(I)$ as follows:

$$f(I) \leq (\sqrt{n})^{\frac{n}{e}+|I|} \quad \text{for all } I \subseteq [\![1, n]\!]. \tag{10}$$

In `enum2s`, we split the enumeration over the interval $[\![1, n]\!]$ into two separate enumerations, which correspond to the intervals $[\![1, \eta]\!]$ and $[\![\eta + 1, n]\!]$. We now prove the following to establish our complexity bound:

**Corollary 1.** *Let $\mathbf{b}_1, \ldots, \mathbf{b}_n$ be a quasi HKZ-reduced basis (as defined in [13]). For any $0 < c < 1$, there exists a dividing index $\eta \in [\![1, n]\!]$ (depending on c) such that*

$$f(I) \leq (\sqrt{n})^{\frac{cn}{e}+|I|} \quad \textit{for all } I \subseteq [\![1, \eta]\!], \tag{11}$$

*and*

$$f(I) \leq 2^{n/2} \cdot (\sqrt{n})^{\frac{(1-c)n}{e}+|I|} \quad \textit{for all } I \subseteq [\![\eta + 1, n]\!]. \tag{12}$$

*Proof.* Among all integers $t \in [\![1, n]\!]$, we choose $\eta$ to be the maximal $t$ satisfying $f(I) < (\sqrt{n})^{cn/e+|I|}$ for all $I \subseteq [\![1, t]\!]$. By the maximality of $\eta$, there exists at least one $I_0 \subseteq [\![1, \eta + 1]\!]$ such that $f(I_0) \geq (\sqrt{n})^{cn/e+|I_0|}$.

For any $I' \subseteq [\![\eta + 2, n]\!]$, we claim $f(I') < (\sqrt{n})^{(1-c)n/e+|I'|}$. Otherwise, since $I_0$ and $I'$ are disjoint, $|I_0 \cup I'| = |I_0| + |I'|$ and by definition of $f$, we have:

$$\begin{aligned} f(I_0 \cup I') &= f(I_0) \cdot f(I') \\ &> (\sqrt{n})^{cn/e+|I_0|} \cdot (\sqrt{n})^{(1-c)n/e+|I'|} \\ &= (\sqrt{n})^{n/e+|I_0\cup I'|}, \end{aligned}$$

contradicting Inequality 10.

Next, since quasi HKZ-reduced bases inherit the key length constraint of LLL-reduced bases, we have $\frac{\|\mathbf{b}_1^\star\|}{\|\mathbf{b}_{\eta+1}^\star\|} \leq 2^{\eta/2}$. For $I \subseteq [\![\eta + 1, n]\!]$ with $\eta + 1 \in I$, let $I = I'' \cup \{\eta + 1\}$, thus $I'' = I/\{\eta + 1\}$. By definition of $f$:

$$f(I) = \frac{\|\mathbf{b}_1\|}{\|\mathbf{b}_{\eta+1}^\star\|} \cdot f(I'');$$

for $\eta+1 \notin I$, $f(I) = f(I/\{\eta+1\})$ trivially. Thus, for any $I \subseteq [\![\eta+1, n]\!]$, we have

$$\begin{aligned} f(I) &\leq \frac{\|\mathbf{b}_1\|}{\|\mathbf{b}_{\eta+1}^\star\|} \cdot f(I/\{\eta + 1\}) \\ &\leq 2^{\eta/2} \cdot (\sqrt{n})^{(1-c)n/e+|I/\{\eta+1\}|} \\ &\leq 2^{n/2} \cdot (\sqrt{n})^{(1-c)n/e+|I|}. \end{aligned}$$

□

Recall that we have established Equation 9 and we have similar conclusion on $T_{\texttt{enum\_tail}}$ from [13]: We already have Eq. 9 and

$$T_{\texttt{enum\_tail}} = \mathcal{P}(n, \log r, \log B) \cdot 2^{O(n)} \cdot \max_{I \subseteq [\![\eta+1, n]\!]} \left( \frac{r^{|I|}}{(\sqrt{n})^{|I|} \prod_{i \in I} \|\mathbf{b}_i^\star\|} \right). \quad (13)$$

Substituting Equation 11 and Equation 12 into these equations, we can then conclude for this subsection that for any $0 < c < 1$, there exists $\eta \in [\![1, n]\!]$ that divides the lattice into two blocks so that

$$T_{\texttt{enum\_head}} = n^{cn/2e + o(n)}$$

and

$$T_{\texttt{enum\_tail}} = n^{(1-c)n/2e + o(n)}.$$

Substituting Equations 11 and 12 into these expressions, the exponential terms simplify to the form of $n^{cn/(2e)}$ and $n^{(1-c)n/(2e)}$ after absorbing polynomial and subexponential factors into the $o(n)$ term. We thus conclude that for any $0 < c < 1$, there exists a dividing index $\eta \in [\![1, n]\!]$ that partitions the basis indices into two intervals, such that:

$$T_{\texttt{enum\_head}} = n^{\frac{cn}{2e} + o(n)} \quad \text{and} \quad T_{\texttt{enum\_tail}} = n^{\frac{(1-c)n}{2e} + o(n)}.$$

### 4.3 Complexity of Find Neighboring

In this subsection, we analyze the complexity of `preprocessing` (Algorithm 6) and `query` (Algorithm 7). Both follow a procedure similar to that of `enumerate`, with the key difference being that we utilize a well-chosen backbone lattice $\mathcal{L}'$ here. Their complexity conclusions differ, but their derivation processes are analogous. Specifically, the analysis can still be conducted using Equation 13 as the foundational framework.

To ensure these processes have single exponential complexity, the backbone lattice $\mathcal{L}'$ must satisfy specific properties related to its Rankin invariant and covering radius. With a well-chosen backbone lattice $\mathcal{L}'$ for the beehives, we can first prove that the total number of vectors in $\mathcal{L}$ iterated over in the inner loops of `query` (Algorithm 7) is $2^{O(n)}$. We then show that the number of points in $\mathcal{L}'$ within a bounded region (e.g., the covering ball) is also bounded by $2^{O(n)}$. Given these two results, the asymptotic complexity of `query` (Algorithm 7) naturally follows as $2^{O(n)}$. By extension, the time complexity of `preprocessing` (Algorithm 6) is likewise $2^{O(n)}$ for each input vector.

From Equation 13, it is clear that for the backbone lattice $\mathcal{L}'$ to be efficiently enumerable in `query`, two conditions must be satisfied. First, its basis should be nearly orthogonal, ensuring that the Gram-Schmidt norms do not decay geometrically, unlike those of random lattices. Second, the enumeration radius must be upper-bounded by $O(\sqrt{n})$ so that the number of output vectors is bounded by $2^{O(n)}$.

To better characterize the sequence of Gram-Schmidt norms, we introduce the definition of the Rankin factor [20]:

**Definition 2 (Rankin Factor, Rankin Invariant).** *Let* $\mathbf{B}$ *be an* $n$*-dimensional basis of a lattice* $\mathcal{L}$*, and let* $j \leq n$*. We define the ratio*

$$\gamma_{n,j}(\mathbf{B}) \;=\; \frac{\mathrm{vol}(\mathbf{B}_{[1,j]})}{\mathrm{vol}(\mathcal{L})^{j/n}} \;=\; \frac{\mathrm{vol}(\mathcal{L})^{(n-j)/n}}{\mathrm{vol}(\pi_{j+1}(\mathcal{L}))}$$

*as the Rankin factor of* $\mathbf{B}$ *with index* $j$*. The Rankin invariant of* $\mathcal{L}$ *with index* $j$*, denoted* $\gamma_{n,j}(\mathcal{L})$*, is defined as the square of the minimal Rankin factor of index* $j$ *over all bases of* $\mathcal{L}$*, i.e.,*

$$\gamma_{n,j}(\mathcal{L}) \;=\; \left(\min_{\mathbf{B} \text{ is a basis of } \mathcal{L}} \gamma_{n,j}(\mathbf{B})\right)^2.$$

The Rankin factor characterizes the degree of orthonormality of a basis and depends on the reduction level of the lattice basis. The Rankin invariant reflects the maximum possible orthonormality achievable by any basis of a lattice. For random lattices, the Rankin invariants are lower-bounded [4,11] by $2^{\Theta(n \log n)}$. Bases with smaller Rankin factors exist only for a much narrower class of lattices. We thus define the notion of near orthonormal basis:

**Definition 3 (Near orthonormal basis).** *A basis* $\mathbf{B}$ *is near orthonormal if and only if its Rankin factors satisfy* $1 \leqslant \gamma_{n,j}(\mathbf{B}) \leqslant 2^{O(n)}$ *for all* $j \in [\![1, n]\!]$.

**Definition 4 (Good backbone lattice).** *A good backbone lattice for a vector set* $\mathcal{V} \subset \mathcal{L}$ *is a lattice* $\mathcal{L}'$ *satisfying:*

1. *it has a near orthonormal basis (see Definition 3);*
2. *its covering radius (Definition 1) satisfies* $\rho = O(\sqrt{n}) \cdot \det(\mathcal{L}')^{1/n}$*;*
3. $\det(\mathcal{L}) = \det(\mathcal{L}')$.

**Theorem 4.** *For a beehive* $\mathcal{B}$ *with a good backbone lattice* $\mathcal{L}'$ *(Definition 4), the* `findAllBeehivesWithin` *operation runs in time* $2^{O(n)}$.

*Proof.* As indicated by Algorithm 5, `findAllBeehivesWithin` solves the closest vector problem (CVP) for the backbone lattice $\mathcal{L}'$ by enumerating all vectors within distance $R = r + \rho(\mathcal{L}')$, where $r$ denotes an upper bound on the distance from the target vector to $\mathcal{L}'$, and $\rho(\mathcal{L}')$ is the covering radius of $\mathcal{L}'$. We first bound $R$ as follows:

$$\begin{aligned}
R &= r + \rho(\mathcal{L}') && (14)\\
&\leq \lambda_1(\mathcal{L}') + \rho(\mathcal{L}') && (\text{since } r \leq \lambda_1(\mathcal{L}') \text{ for CVP})\\
&\leq \gamma_n \cdot \det(\mathcal{L}')^{1/n} + \rho(\mathcal{L}') && (\text{by properties of } \lambda_1 \text{ and lattice determinants})\\
&= O(\sqrt{n}) \cdot \det(\mathcal{L}')^{1/n}, && (\text{by } \rho(\mathcal{L}') = O(\sqrt{n}) \cdot \det(\mathcal{L}')^{1/n} \text{ from the theorem})
\end{aligned}$$

where $\lambda_1(\mathcal{L}')$ denotes the minimal norm of non-zero vectors in $\mathcal{L}'$.

The time complexity of `findAllBeehivesWithin` is bounded by the number of nodes in the enumeration, which is characterized by Equation 13. For the basis $\{\mathbf{b}_i\}$ of $\mathcal{L}'$, we analyze the key term in the enumeration bound:

$$\begin{aligned}
&\max_{I\subseteq[\![1,n]\!]} \left(\frac{R^{|I|}}{n^{|I|/2}\prod_{i\in I}\|\mathbf{b}_i^\star\|}\right) \\
&= \max_{I\subseteq[\![1,n]\!]} \left(\frac{R}{\sqrt{n}\cdot\det(\mathcal{L}')^{1/n}}\right)^{|I|} \cdot \frac{(\det(\mathcal{L}'))^{|I|/n}}{\prod_{i\in I}\|\mathbf{b}_i^\star\|} \\
&= \max_{I\subseteq[\![1,n]\!]} O(1)^{|I|} \cdot \frac{(\det(\mathcal{L}'))^{|I|/n}}{\prod_{i\in I}\|\mathbf{b}_i^\star\|} && \text{(by } R = O(\sqrt{n}\cdot\det(\mathcal{L}')^{1/n})) \\
&= \max_{I\subseteq[\![1,n]\!]} O(1)^{|I|} \cdot \gamma_{n,|I|}(\mathbf{B}) && \text{(by Rankin factor definition)} \\
&= 2^{O(n)}. && \text{(since } \mathcal{L}' \text{ is near orthonormal bases (Definition 3))}
\end{aligned}$$

Thus, the time complexity of `findAllBeehivesWithin` is $2^{O(n)}$.

□

**Theorem 5.** *For a beehive $\mathcal{B}$ with a good backbone lattice $\mathcal{L}'$ (Definition 4), the* `query` *operation runs in time $2^{O(n)}$.*

*Proof.* Theorem 4 indicates that both running time and the output size of the `findAllBeehivesWithin` operation are $2^{O(n)}$, which bounds the outer loop of `query` (Algorithm 7). We now move to count the number of inner loops, which is upper bounded by the number of vectors in lattice $\mathcal{L}$ within radius $R = 2\rho(\mathcal{L}) + \gamma_n \cdot \det(\mathcal{L})^{1/n}$. Similar to Equation 14 we have $R = O(\sqrt{n}) \cdot \det(\mathcal{L})^{1/n}$. We can therefore corresponds the number of vectors to $N_1$ in Equation 8.

$$\begin{aligned}
N_1 &= 2^{O(n)} \cdot \prod_{i\in[\![1,n]\!]} \frac{R}{\sqrt{n}\|\mathbf{b}_i^\star\|} \\
&= 2^{O(n)} \cdot \frac{R^n}{n^{n/2}\cdot\det(\mathcal{L})} \\
&= 2^{O(n)}
\end{aligned}$$

Therefore the time complexity of `query` (Algorithm 7) is $2^{O(n)}$. □

Finally, we conclude the following:

**Theorem 6.** *For a beehive $\mathcal{B}$ with a good backbone lattice $\mathcal{L}'$ (Definition 4), the* `preprocessing` *operation runs in time $2^{O(n)}$ per vector.*

*Proof.* The complexity of the procedure `findClosestBeehive` can be trivially reduced to that of `findAllBeehivesWithin(`$\rho$`, u)`, thus it is also of time complexity $2^{O(n)}$. Therefore `preprocessing` is also of time complexity $2^{O(n)}$. □

### 4.4 Existence of Good Backbone Lattices

To complete the complexity analysis, this final subsection demonstrates the existence of good backbone lattices by presenting feasible candidate options, thereby providing a basis for validating the earlier complexity bounds.

We recall the following key lattices [9]:

- $\mathbb{Z}^n \subset \mathbb{R}^n$: All integer coordinate vectors $(x_1, \ldots, x_n) \in \mathbb{Z}^n$ .
- $\mathbf{A}_n \subset \mathbb{R}^{n+1}$: $\left\{(x_1, \ldots, x_{n+1}) \in \mathbb{Z}^{n+1} \mid \sum x_i = 0\right\}$ .
- $\mathbf{D}_n \subset \mathbb{R}^n$ $(n \geq 4)$: $\{(x_1, \ldots, x_n) \in \mathbb{Z}^n \mid \sum x_i \text{ even}\}$.

| **Backbone Lattice** | $\det(\mathcal{L})$ | **Covering Radius** $\rho$ | $\sup_i \gamma_{n,i}$ |
|---|---|---|---|
| $\mathbb{Z}^n$ | 1 | $\sqrt{n}/2$ | 1 |
| $\mathbf{A}_n$ | $\sqrt{n+1}$ | $\sqrt{\frac{a(n+1-a)}{n+1}}$ $(a = \lfloor \frac{n+1}{2} \rfloor)$ | $O(n)$ |
| $\mathbf{D}_n$ $(n \geq 4)$ | 2 | $\sqrt{n}/2$ | $O(n)$ |

Table 3: Candidates for Good Backbone Lattices $\mathcal{L}'$

*Note:* The supremum of the Rankin invariant, $\sup_i \gamma_{n,i}$, quantifies the degree of orthonormality of a lattice.

Given a lattice $\mathcal{L}$ of dimension $\eta$, take $\mathcal{G}$ to be any of the above candidate backbone lattice of the same dimension, then

$$\mathcal{L}' = \left(\frac{\det(\mathcal{L})}{\det(\mathcal{G})}\right)^{1/\eta} \cdot \mathcal{G}$$

is a good backbone lattice. This finishes the proof for the chief complexity conclusion in Table 2.

## 5 Heuristic Analysis and Practical Adaptation

In the previous sections, we established the main complexity results. Here, we focus on the practical aspects of the algorithm. First, we show how to adjust our enumeration procedure to match pruned enumeration, which is the most widely used practical variant of enumeration. Then, we discuss the practical complexity of the neighbor-finding algorithm.

Throughout this section, we will need *Gaussian Heuristic* for an evaluation of the practical cost of running time. It provides an estimate on the number of lattice points inside a set.

**Definition 5 (Gaussian Heuristic 1).** *Given a lattice $\mathcal{L}$ and a set $\mathcal{S}$, the number of points in $\mathcal{S} \cap \mathcal{L}$ is approximately* $\mathrm{vol}(\mathcal{S})/\mathrm{vol}(\mathcal{L})$.

**Definition 6 (Gaussian Heuristic 2).** *We denote by* $\mathrm{gh}(\mathcal{L})$ *the expected first minimum of a lattice* $\mathcal{L}$ *according to the Gaussian Heuristic. For a full-rank lattice* $\mathcal{L} \subset \mathbb{R}^n$*, it is given by:*

$$\mathrm{gh}(\mathcal{L}) = \sqrt{\frac{n}{2\pi e}} \cdot \mathrm{Vol}(\mathcal{L})^{1/n}.$$

*We also denote* $\mathrm{gh}(n)$ *for* $\mathrm{gh}(\mathcal{L})$ *of any* $n$*-dimensional lattice* $L$ *of volume* 1*:*

$$\mathrm{gh}(n) = \sqrt{\frac{n}{2\pi e}}.$$

Under the Gaussian Heuristic, we can predict the shape $0, \ldots, n-1$ of an BKZ-reduced basis, i.e., the sequence of expected norms for the vectors $\mathbf{b}_i^*$. The sequence is inductively defined as follows:

**Definition 7 (Geometric Series Assumption).** *Let* $\mathbf{B}$ *be a BKZ-*$\beta$ *reduced basis of a lattice of volume* 1*. The Geometric Series Assumption states that:*

$$\|\mathbf{b}_i^*\| = \alpha_\beta^{\frac{n+1}{2}-i}$$

*where* $\alpha_\beta = \mathrm{gh}(\beta)^{2/\beta}$*.*

*This model is reasonably accurate in practice for* $\beta > 50$ *and* $\beta \ll n$*. For further discussion on this model and its accuracy, the reader may refer to [8,7,25].*

### 5.1 Pruned Enumeration

We first clarify how pruned enumeration differs from full enumeration: instead of visiting all nodes within the bound $r$ in the sublattice (as described in Line 9 of Algorithm 1), pruned enumeration reduces computational costs by skipping certain tree branches and using smaller bounds for different sublattices. This idea originated with Schnorr and Euchner [22] and was further developed by Gama, Nguyen, and Regev through the concept of extreme pruning [12].

Formally, it modifies the $n$ inequalities $\|\pi_{n+1-k}(v)\| \leq r$ (from full enumeration) by replacing $r$ with a vector $\mathbf{r} = (r_1, \ldots, r_n)$, where $r_1 \leq \cdots \leq r_n = r$ are real numbers determined by the specific pruning strategy. In other words, each of the $n$ inequalities in Equation 1 is adjusted to $\|\pi_{n+1-k}(v)\| \leq r_k$. A corresponding algorithm description is given in Appendix A, with the difference highlighted. Below, we demonstrate how to adapt our enumeration procedure to align with this pruned approach.

For both the enumeration procedure in head block and in the tail block, we can directly transport the change from $r$ to the pruning strategy $\mathbf{r}$ to the original enumeration procedure described in Algorithm 1 and 4. This way, the pruned enumeration for the tail block follows the same procedure, and therefore possesses the same cost analysis, which consists of estimating the number of nodes in enumeration using the Gaussian Heuristic. For the head block, on the other hand, the region defined by the bounding strategy $\mathbf{r}$ is augmented by a

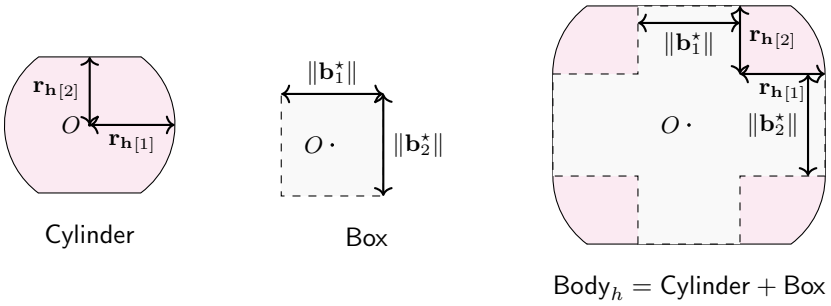



Fig. 5: The shape of $\mathsf{Body}_h$ to ensure $(\mathsf{Body}_h - \mathsf{Box}) \subseteq \mathsf{Cylinder}$

supplementary Babai-box ($\mathsf{Box}_h$, and therefore we need to compute its volume before applying the Gaussian Heuristic.

We use $\mathsf{Cylinder}_h$ to represent the cylinder intersection related to the original pruned enumeration procedure defined by the bounding strategy $\mathbf{r}_h = \mathbf{r}_{[1,\eta]}$, and $\mathsf{Box}_h$ to denote the Babai box of the lattice basis $\mathbf{B}_{[1,\eta]}$.

The total volume $\mathsf{Body}_h$ we are enumerating can be expressed as the Minkowski sum $\mathsf{Box}_h + \mathsf{Cylinder}_h$, defined as $\mathsf{Body}_h = \{\mathbf{v} + \mathbf{v}' \mid \mathbf{v} \in \mathsf{Box}_h \wedge \mathbf{v}' \in \mathsf{Cylinder}_h\}$. An illustration is presented in Figure 5.

The volume of $\mathsf{Body}_h$ can be computed by using the following theorem.

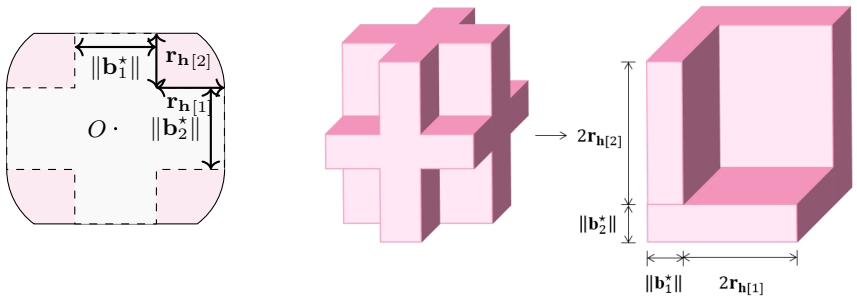
**Theorem 7.** *The volume of the enumerated space* $\mathsf{Body}_h$ *can be computed by*

$$\mathrm{Vol}(\mathsf{Body}_h) = \mathrm{Vol}(\mathsf{Cylinder}) + \prod_{i=1}^{\eta}(2\mathbf{r_h}_{[i]} + \|\mathbf{b}_i^\star\|) - \prod_{i=1}^{\eta}(2\mathbf{r_h}_{[i]}). \tag{15}$$

*Proof.* As shown in (a) of Figure 6, the volume of $\mathsf{Body}_h$ is the sum of the volumes of the pink region and the gray region. The volume of the pink region is $\mathrm{Vol}(\mathsf{Cylinder})$. Furthermore, the gray region is shown in the middle part of Figure 6; we transform it to the right side of the figure for clarity. The volume of the gray region can be derived from the difference between two rectangular volumes: $\prod_{i=1}^{\eta}(2r_{h,[i]} + \|\mathbf{b}_i^\star\|)$ and $\prod_{i=1}^{\eta}(2r_{h,[i]})$. Summing the volumes of the pink cylinder and the gray region, we arrive at the conclusion stated in the theorem. □

### 5.2 Finding Neighboring Vectors

In Table 3, we list several potential choices for backbone lattices. For simplicity, we select $\mathbb{Z}^n$ for cost estimation herein. $\mathbb{Z}^n$ has the favorable property that its standard basis is perfectly orthonormal, and thus for many problems it is easy to decompose into subspaces of dimension 1. This allows us to design particularly efficient algorithms for this backbone lattice.




(a) $\mathsf{Box}_h + \mathsf{Cylinder}_h$. (b) The gray area and its transformation.

Fig. 6: The volume transformation in the proof of the Theorem 7.

Let the head block be $\mathbf{B}_h = \mathbf{B}_{[1,\eta]}$. On the subspace $\mathrm{span}(\mathbf{B}_h)$, we deploy the backbone lattice $\mathcal{L}' = \det(\mathbf{B}_h)^{1/\eta} \cdot \mathbb{Z}^\eta = \left\{\det(\mathbf{B}_h)^{1/\eta} \cdot \mathbf{x} \mid \mathbf{x} \in \mathbb{Z}^\eta\right\}$. The orthonormality of this backbone lattice's standard basis allows for far more efficient `preprocess` and `query` algorithms than the general approach presented in the theoretical proof of the previous chapter. We analyze the practical complexity of this design in the following.

### 5.2.1 Optimizing `preprocess` and Its Complexity Estimation

In this particular case of using the rescaled $\mathbb{Z}^n$ (denoted as $\mathcal{L}'$) as the backbone lattice, the `findClosestBeehive`$(\mathbf{u})$ can be trivially reduced to solving subproblems in dimension 1. Let $\lfloor x \rceil$ denote rounding $x$ to the nearest integer, and let $g = \det(\mathbf{B}_h)^{1/\eta}$. Then,

$$\mathbf{p} = (\lfloor u_i/g \rceil \cdot g)_{i=1}^{\eta} \in \mathcal{L}'$$

is the vector in $\mathcal{L}'$ closest to $\mathbf{u}$. Since this involves $\eta$ independent 1-dimensional rounding operations, the running time is $O(\eta)$.

### 5.2.2 Optimizing `query` and Its Complexity Estimation

Again, when the backbone lattice $\mathcal{L}'$ is the rescaled $\mathbb{Z}^n$, there is a simplified algorithm for `query`.

In the `query(u, r)` process, we aim to visit all beehives intersecting the ball $\mathsf{Ball}_{\mathbf{u},r}$, centered at $\mathbf{u}$ with radius $r$. We can reformulate this in graph-theoretic terms: each beehive is a node, and two beehives are connected by an edge if and only if they are direct neighbors. That is to say, their centers $\mathbf{p}$ and $\mathbf{p}'$ differ in exactly one coordinate, say index $i$, and $\mathbf{p}_i$ and $\mathbf{p}'_i$ are consecutive values. This forms a homogeneous graph $\mathcal{G}$ with degree $2\eta$.

Let $\mathsf{Grid}_{\mathbf{p}}$ be the beehive with $\mathbf{p}$ as its center. Then the point maximizing the distance to $\mathbf{p}$ within $\mathsf{Ball}(\mathbf{u}, r)$ is

$$\mathbf{t} = \mathbf{u} + \frac{r}{\|\mathbf{u} - \mathbf{p}\|} \cdot (\mathbf{u} - \mathbf{p}), \tag{16}$$

which lies on the boundary of $\mathsf{Ball}(\mathbf{u}, r)$, see Figure 7. In other words, the condition $\mathsf{Grid}_{\mathbf{p}} \cap \mathsf{Ball}(\mathbf{u}, r) \neq \emptyset$ is equivalent to $\mathbf{t} \in \mathsf{Grid}_{\mathbf{p}}$.

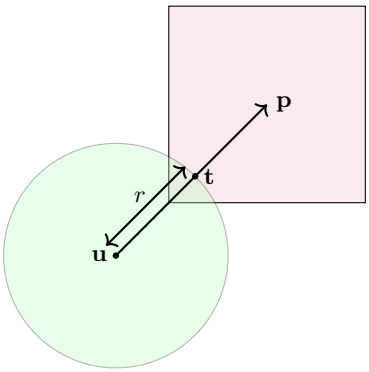



Fig. 7: $\mathbf{t}$ is the point minimizing distance to $\mathbf{p}$ within $\mathsf{Ball}_{\mathbf{u},r}$. If $\mathbf{t}$ is in $\mathsf{Grid}_{\mathbf{p}}$ then $\mathsf{Grid}_{\mathbf{p}} \cap \mathsf{Ball}_{\mathbf{u},r} \neq \emptyset$

Furthermore, if $\mathsf{Grid}(\mathbf{p}_0)$ is the beehive containing $\mathbf{u}$, and $\mathsf{Grid}(\mathbf{p}_m)$ is a beehive intersecting $\mathsf{Ball}_{\mathbf{u},r}$, then $\mathbf{p}_0$ and $\mathbf{p}_m$ are connected in $\mathcal{G}$ via a Manhattan path. See Figure 8 for illustration. Such a path moves along one coordinate at a time, consistent with the graph's edge definition, ensuring a direct connection between the two nodes. This means we can visit all beehives intersecting $\mathsf{Ball}_{\mathbf{u},r}$ (i.e., with non-zero intersections) by a tree traversal algorithm, either depth-first search or breadth-first search, which starts at $\mathbf{p}_0$. See Algorithm 8 for details.

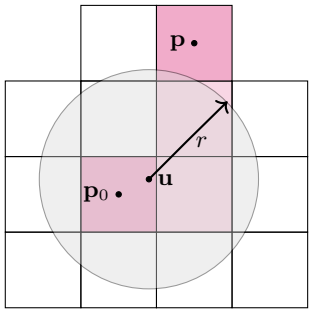



Fig. 8: If $\mathbf{u} \in \mathsf{Grid}_{\mathbf{p}_0}$ and $\mathsf{Grid}_{\mathbf{p}} \cap \mathsf{Ball}_{\mathbf{u},r} \neq \emptyset$, then there exists a Manhattan path (highlighted in pink) connecting $\mathsf{Grid}_{\mathbf{p}_0}$ and $\mathsf{Grid}_{\mathbf{p}}$.

The complexity of Algorithm 8 is $O(\mathcal{R} + E(\mathcal{R}))$ where $E$ is the size of the edges connected to $\mathcal{R}$. Since $E \leq 2\eta \cdot \|\mathcal{R}\|$, we conclude that the time and space complexity are both $O(\mathcal{R})$.

**Algorithm 8** `findAllBeehivesWithin_tree`

**Input:** search radius $r \in \mathbb{R}$, and a vector $\mathbf{u} \in \mathbb{R}^n$.
**Output:** a set of vectors $\mathcal{S} = \{\mathbf{p} \in \mathbb{Z}^n \wedge \|\mathbf{p} - \mathbf{u}\| \leq r\}$

$\mathcal{S} \leftarrow \{$`findClosestBeehive`$(\mathbf{u})\}$.
$\mathcal{V} \leftarrow \mathcal{S}; \mathcal{R} \leftarrow \mathcal{S}$;
**while** $\mathcal{S} \neq \emptyset$ **do**
    $\mathbf{w} \leftarrow \mathcal{S}$.pop()
    $\mathcal{R} \leftarrow \mathcal{R} \cup \{\mathbf{w}\}$ ▷ Result set
    **for** each $v \in w$.neighbor() **do**
        **if** $\mathsf{Grid}_{\mathbf{v}} \cap \mathsf{Ball}_{\mathbf{u},r} \neq \emptyset$ and $\mathbf{v} \notin \mathcal{V}$ **then** ▷ See Equation 16
            $\mathcal{S} \leftarrow \mathcal{S} \cup \{\mathbf{v}\}$ ▷ To be expanded
            $\mathcal{V} \leftarrow \mathcal{V} \cup \{\mathbf{v}\}$ ▷ Mark as visited
        **end if**
    **end for**
**end while**
**return** $\mathcal{R}$

It remains to estimate the output size of Algorithm 8. To this end, we again employ the Gaussian Heuristic and Geometric Series Assumption (see Definitions 5 and 7), and for simplicity we rescale the whole lattice to have determinant 1. Then the volume of the grid is:

$$\begin{aligned}\det(\mathcal{L}') &= \prod_{i=1}^{\eta} \|\mathbf{b}_i^\star\| = \prod_{i=1}^{\eta} \alpha_\beta^{\frac{\eta+1}{2}-i} \\ &= \alpha_\beta^{\frac{\eta(\eta-\eta)}{2}}\end{aligned}$$

where $\beta$ denotes the block size for which the lattice $\mathcal{L}'$ is a priori BKZ-$\beta$ reduced, and $\mathrm{gh}(\beta)$ is the Gaussian Heuristic constant ($\mathrm{gh}(\beta) = \sqrt{\beta/(2\pi e)}$), thus $\alpha_\beta = \mathrm{gh}(\beta)^{2/\beta} > 1$ in practice. It follows that $\det(\mathcal{L}') > 1$. Meanwhile, the query radius $r$ is the square root of the sum of the first $\eta$ coordinates of the shortest vector in $\mathcal{L}$; thus, we have $r = \sqrt{\frac{\eta}{2\pi e}}$, and the volume of $\mathsf{Ball}(0, r)$ is 1 by the Gaussian Heuristic. Since this volume is much smaller than that of $\mathcal{L}'$, we heuristically estimate that the number of nodes $\#V(\mathcal{G})$ is very small and at most $2^\eta$.

Indeed, Mazo et al. [17] study the number of integral points within $n$-dimensional balls and state that when the radius $r$ is of the form $r = \alpha \cdot \sqrt{n}$, the number of integral points inside this ball is approximately $2^{c\eta}$ for some constant $c < 1$ depending on $\alpha$.[7] Specifically, they list a lookup table for the relation between $\alpha$ and $c$. In our case, $\alpha$ is given by $\alpha = \frac{r}{\alpha_\beta^{(\eta-n)/2}} = \frac{\sqrt{\eta/(2\pi e)}}{\alpha_\beta^{(\eta-n)/2}}$, and the related $c$ can be estimated using the table provided by [17].

[7] This is a well-known example of violation of the Gaussian Heuristic (GH).

This already represents a significant improvement in efficiency compared to the original proven method, which requires iterating over all beehives whose centers lie within distance $\rho + r$ of $\mathbf{u}$.

### 5.3 Paramter Optimization and Cost Estimation

Once the backbone lattice is fixed as $\mathcal{L}' = \det(\mathbf{B}_h)^{1/\eta} \cdot \mathbb{Z}^\eta$, the core parameters of the algorithm reduce to the pruning vector $\mathbf{r}$ and the lattice block division index $\eta$. This section focuses on optimizing these parameters: we first adopt a predefined pruning strategy for $\mathbf{r}$, then iterate over all feasible values of $\eta$ to minimize the total enumeration cost. In our simplified model, the costs of `query` and `preprocessing` are estimated to be negligible and thus omitted.

Additionally, we assume the running time $T$ is proportional to the number of enumeration nodes, with a practical scaling factor of $2^{26}$ nodes per second. Thus, we use the estimated number of nodes as a proxy for $T$ in the following analysis.

$$T_{\texttt{enum2s}} = \min_{\eta} \left( T_{\text{enum\_head}} + T_{\text{enum\_tail}} \right).$$

To verify the practical optimization effect of the proposed algorithm, we conducted experiments on lattice bases of typical dimensions, which are standard for evaluating enumeration-based lattice methods. For each predefined pruning vector $\mathbf{r}$, we iterated over all feasible values of $\eta$ (ranging from 0 to the lattice dimension $n$) and computed the corresponding $T_{\text{enum\_head}}$ and $T_{\text{enum\_tail}}$ at each $\eta$. To better visualize the cost variation trend, the $y$-axis uses a $\log_2$-based scale.

Table 4: Enumeration Cost Evaluation for BKZ-50 Reduced Basis of Different Lattice Dimensions

| $p_{\text{succ}}$ | 0.04 | 0.11 | 0.26 | 0.48 | 0.72 |
|---|---|---|---|---|---|
| $T_0$ ($\log_2$ base) | 34.71 | 37.4 | 40.16 | 42.74 | 45.18 |
| $T_{\texttt{enum2s}}$ ($\log_2$ base) | 31.76 | 33.85 | 36.49 | 39.01 | 41.37 |
| Optimal $\eta$ | 43 | 43 | 43 | 43 | 43 |

(a) Lattice dimension $N = 80$

| $p_{\text{succ}}$ | 0.01 | 0.03 | 0.09 | 0.21 | 0.40 |
|---|---|---|---|---|---|
| $T_0$ ($\log_2$ base) | 38.64 | 41.47 | 44.28 | 47.04 | 49.72 |
| $T_{\texttt{enum2s}}$ ($\log_2$ base) | 35.34 | 38.16 | 40.88 | 43.53 | 46.07 |
| Optimal $\eta$ | 50 | 49 | 50 | 50 | 50 |

(b) Lattice dimension $N = 90$

The experimental results for $N = 80, 90$ are summarized in Table 4. It can be observed that the practical advantage of $T_{\texttt{enum2s}}$ over $T_0$ remains relatively modest. For example, with $N = 80$, the cost reduction is confined to a range of

$2^3$ to $2^4$—far from the theoretically expected square-root scale. This suboptimal performance stems from the additional volume introduced in $T_{\text{enum\_head}}$, which causes its growth rate to expand exponentially.

Figure 9 details this trend, revealing a consistent pattern across different lattice dimensions and pruning vectors: as $\eta$ increases, $T_{\text{enum\_head}}$ exhibits a monotonic upward trend. This is because a larger $\eta$ expands the dimension range of the head enumeration, increasing the number of candidate vectors to process. In contrast, $T_{\text{enum\_tail}}$ decreases monotonically with $\eta$: a larger $\eta$ reduces the remaining dimension range for the tail enumeration, thereby lowering its computational burden. The two curves $T_{\text{enum\_head}}$ and $T_{\text{enum\_tail}}$ intersect slightly to the right of the midpoint of the $\eta$-axis; this intersection corresponds to the minimum of the total enumeration cost $T_{\texttt{enum2s}} = T_{\text{enum\_head}} + T_{\text{enum\_tail}}$, marking the optimal $\eta$ configuration for the given pruning vector $\mathbf{r}$ and lattice dimension. More figures are given in Appendix B for interested reader.

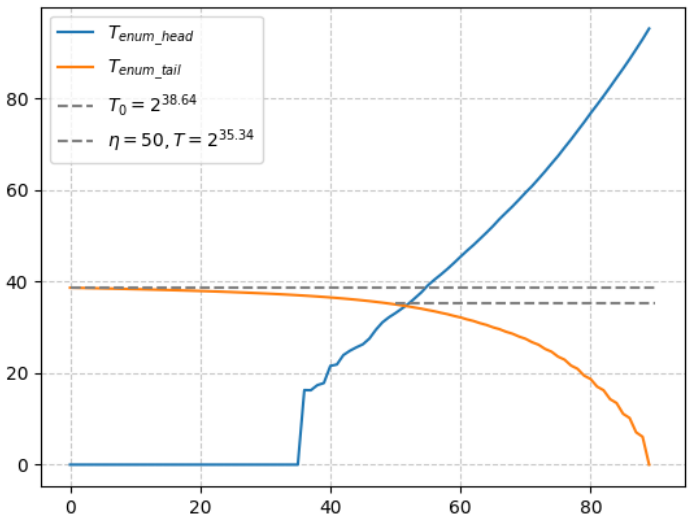



Fig. 9: Cost Evaluation for $N = 90$ and $p = 0.01$

## 6 Discussion

This paper proposes a memory-time tradeoff technique for the Shortest Vector Problem (SVP). While our current analysis shows it yields only modest improvements to enumeration performance in practice, it offers a new perspective

on the "space-for-time" tradeoff from an algorithmic standpoint—one that can be integrated into existing design frameworks for SVP enumeration methods.

In current SVP algorithm landscape, there are methods that improve solution dimension by concatenating head lattice vectors like the technique proposed by "Dimension for Free" approach [10]. In this section, we aim to connect our proposed algorithm with these existing approaches by identifying their underlying commonalities in lattice computation optimization, and compare the differences between them so as to open up new possibilities for future research.

### 6.1 Relationship with Dimension for Free

The dimension-for-free technique introduced by Ducas [10] extends the short vectors visited while sieving in dimension $n$ to a higher dimension of approximately $n + \Theta(n/\log n)$. Specifically, they apply a `lift` operation to every short vector encountered in the sieving process, once a sufficient number of short vectors are visited, one of them will become the shortest vector after the `lift` step.

In our approach, by contrast, we do not only consider vectors lifted to a higher dimension, we also check points within a certain range. This ensures the exact closest vector is found, enabling us to extend the dimension of the head block to a higher value.

From Figure 9, we observe that for the first few values of $\eta$, the predicted number of vectors to be visited ($T_{\text{enum\_head}}$) is 1. This aligns with the estimations in [10], which note that for the initial few blocks, short vectors can be extended to find their closest neighbors with the minor cost of a `lift` operation. However, as the dimension of the head block increases, the Babai parallelepiped can become highly elongated, since random lattice bases conform to the Geometric Series Assumption (GSA, Definition 7). For this reason, we need to query more neighboring vectors instead of only considering the lifted vector, as illustrated in Figure 10.

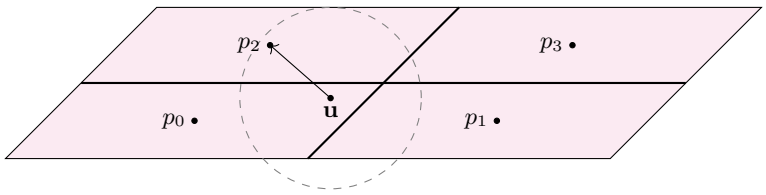



Fig. 10: When the parallelepipeds have elongated shapes, the closest vector to a query vector $\mathbf{u}$ is not necessarily the center vector of the parallelepiped it lies in.

### 6.2 Future Work

**1. Parameter Optimization:** The optimization of the pruning vector $\mathbf{r}$ for our proposed algorithm is more complex than that for simple enumeration al-

gorithms, as the success probability estimation becomes intricate. Currently, we use the trimmed pruning vector optimized for simple enumeration, but the resulting change in success probability remains unclear. A deeper understanding of this optimization process could improve the practical efficiency of our algorithm.

2. **Adaptation to Other Algorithms:** As discussed in the previous subsection, our work can be viewed as an extension of the dimension-for-free technique in certain aspects. It is therefore promising to explore whether this technique can be used to extend the head block of sieving algorithms, thereby gaining additional free dimensions. Currently, the primary bottleneck limiting sieving algorithms from solving higher-dimensional SVP is their high memory requirement. It is therefore a natural goal to explore the feasibility of increasing the head block dimension.

## References


1. Ajtai, M., Kumar, R., Sivakumar, D.: A sieve algorithm for the shortest lattice vector problem. In: Vitter, J.S., Spirakis, P.G., Yannakakis, M. (eds.) STOC 2001. pp. 601–610. https://doi.org/10.1145/380752.380857
2. Becker, A., Ducas, L., Gama, N., Laarhoven, T.: New directions in nearest neighbor searching with applications to lattice sieving. In: Krauthgamer, R. (ed.) SODA 2016. pp. 10–24. https://doi.org/10.1137/1.9781611974331.ch2
3. Becker, A., Gama, N., Joux, A.: Solving shortest and closest vector problems: The decomposition approach. IACR Cryptol. ePrint Arch. p. 685 (2013), http://eprint.iacr.org/2013/685
4. Becker, A., Gama, N., Joux, A.: A sieve algorithm based on overlattices. LMS Journal of Computation and Mathematics **17**(A), 49–70 (2014)
5. Becker, A., Gama, N., Joux, A.: Speeding-up lattice sieving without increasing the memory, using sub-quadratic nearest neighbor search. IACR Cryptol. ePrint Arch. p. 522 (2015), http://eprint.iacr.org/2015/522
6. Becker, A., Laarhoven, T.: Efficient (ideal) lattice sieving using cross-polytope LSH. In: Pointcheval, D., Nitaj, A., Rachidi, T. (eds.) AFRICACRYPT 2016. pp. 3–23. https://doi.org/10.1007/978-3-319-31517-1_1
7. Chen, Y.: Réduction de réseau et sécurité concrete du chiffrement completement homomorphe. Ph.D. thesis, Paris 7 (2013)
8. Chen, Y., Nguyen, P.Q.: BKZ 2.0: Better lattice security estimates. In: Lee, D.H., Wang, X. (eds.) ASIACRYPT 2011. pp. 1–20. https://doi.org/10.1007/978-3-642-25385-0_1
9. Conway, J.H., Sloane, N.J.A.: Sphere packings, lattices and groups, vol. 290. Springer Science & Business Media (2013)
10. Ducas, L.: Shortest vector from lattice sieving: A few dimensions for free. In: Nielsen, J.B., Rijmen, V. (eds.) EUROCRYPT 2018. pp. 125–145. https://doi.org/10.1007/978-3-319-78381-9_5
11. Gama, N., Howgrave-Graham, N., Koy, H., Nguyen, P.Q.: Rankin's constant and blockwise lattice reduction. In: Annual International Cryptology Conference. pp. 112–130. Springer (2006)
12. Gama, N., Nguyen, P.Q., Regev, O.: Lattice enumeration using extreme pruning. In: EUROCRYPT 2010. pp. 257–278. https://doi.org/10.1007/978-3-642-13190-5_13

13. Hanrot, G., Stehlé, D.: Improved analysis of kannan's shortest lattice vector algorithm. In: Menezes, A. (ed.) CRYPTO 2007. pp. 170–186. Springer (2007), https://doi.org/10.1007/978-3-540-74143-5_10
14. Kannan, R.: Improved algorithms for integer programming and related lattice problems. In: STOC 1983. pp. 193–206. https://doi.org/10.1145/800061.808749
15. Laarhoven, T.: Search problems in cryptography: from fingerprinting to lattice sieving (2016), https://pure.tue.nl/ws/portalfiles/portal/14673128/20160216_Laarhoven.pdf
16. Martinet, J.: Perfect lattices in Euclidean spaces, vol. 327. Springer Science & Business Media (2013)
17. Mazo, J.E., Odlyzko, A.M.: Lattice points in high-dimensional spheres. Monatshefte für Mathematik **110**(1), 47–61 (1990)
18. Micciancio, D., Voulgaris, P.: Faster exponential time algorithms for the shortest vector problem. In: Charikar, M. (ed.) SODA 2010. pp. 1468–1480. https://doi.org/10.1137/1.9781611973075.119
19. Nguyen, P.Q., Vidick, T.: Sieve algorithms for the shortest vector problem are practical. J. Math. Cryptol. **2**(2), 181–207 (2008), https://doi.org/10.1515/JMC.2008.009
20. Rankin, R.A.: On positive definite quadratic forms. Journal of the London Mathematical Society **1**(3), 309–314 (1953), https://doi.org/10.1112/jlms/s1-28.3.309
21. Schnorr, C., Euchner, M.: Lattice basis reduction: Improved practical algorithms and solving subset sum problems. Math. Program. **66**, 181–199 (1994), https://doi.org/10.1007/BF01581144
22. Schnorr, C., Hörner, H.H.: Attacking the chor-rivest cryptosystem by improved lattice reduction. In: Guillou, L.C., Quisquater, J. (eds.) EUROCRYPT 1995. pp. 1–12. Springer (1995), https://doi.org/10.1007/3-540-49264-X_1
23. Shoup, V., et al.: NTL: a library for doing number theory (2001), available at https://libntl.org
24. The FPLLL development team: fplll, a lattice reduction library, Version: 5.4.5 (2023), available at https://github.com/fplll/fplll
25. Yu, Y., Ducas, L.: Second order statistical behavior of lll and bkz. In: International Conference on Selected Areas in Cryptography. pp. 3–22. Springer (2017)

# A Pruned Enumeration Algorithm

# B More Cost Evaluation

Appendix B includes supplementary plots for the predefined pruning vectors $\mathbf{r}$ from Table 4—ones not visualized in Figure 9 due to space constraints. Following the same format (with a $\log_2$-scaled $y$-axis), each plot shows the monotonic trends of $T_{\text{enum\_head}}$ (upward) and $T_{\text{enum\_tail}}$ (downward) with $\eta$, along with their intersection marking the optimal $\eta$ for the corresponding $\mathbf{r}$.

**Algorithm 9** `pruned_enumerate`

**Input:** a lattice basis $\mathbf{B} = (\mathbf{b}_1, \ldots, \mathbf{b}_n)$, search radius $r \in \mathbb{R}$, a bounding vector $\mathbf{r} = (r_1, \ldots, r_n)$.
**Output:** a set of vectors $\mathcal{S} = \{\mathbf{v} \in \mathcal{L} \wedge \|\mathbf{v}\| \leq r\}$

Compute $\mu_{i,j}$'s and $\|\mathbf{b}_i^\star\|^2$'s.
$\mathbf{u} \leftarrow \mathbf{0}; \mathbf{l} \leftarrow \mathbf{0}; \mathcal{S} \leftarrow \emptyset; i \leftarrow 1$
**while** $i \leq n$ **do**
    $l_i \leftarrow (u_i + \sum_{j>i} u_j \mu_{j,i})^2 \|\mathbf{b}_i^\star\|^2$
    **if** $\sum_{j>i} l_j \leq r^2$ **then** ▷ If within $r^2$ bound
        **if** $i = 1$ **then**
            $\mathcal{S} \leftarrow \mathcal{S} \cup \{\mathbf{u} \cdot \mathbf{B}\}$ ▷ One more solution found
        **else**
            $i \leftarrow i - 1$; $u_i \leftarrow \left\lceil -\sum_{j>i}(x_j \mu_{j,i}) - \sqrt{\frac{r_i^2 - \sum_{j>i} l_j}{\|\mathbf{b}_i^\star\|^2}} \right\rceil$ ▷ Go down the tree
        **end if**
    **else**
        $i \leftarrow i + 1$; $u_i \leftarrow u_i + 1$ ▷ Beyond $r^2$ bound, go up the tree
    **end if**
**end while**
**return** $\mathcal{S}$

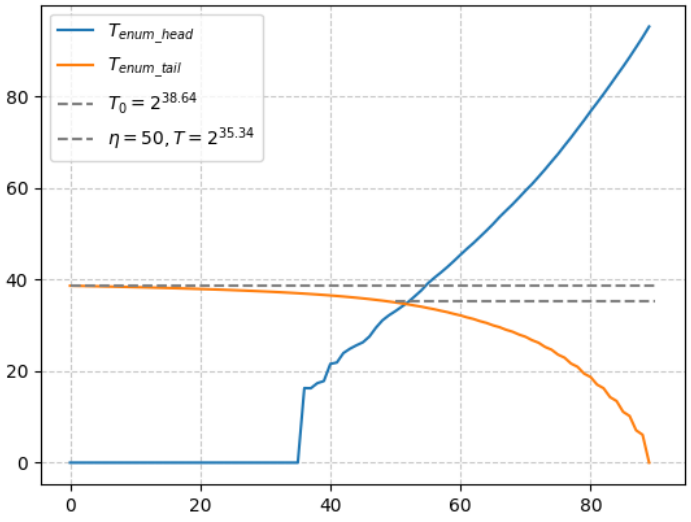



Fig. 11: Cost Evaluation for $N = 90$ and $p = 0.01$

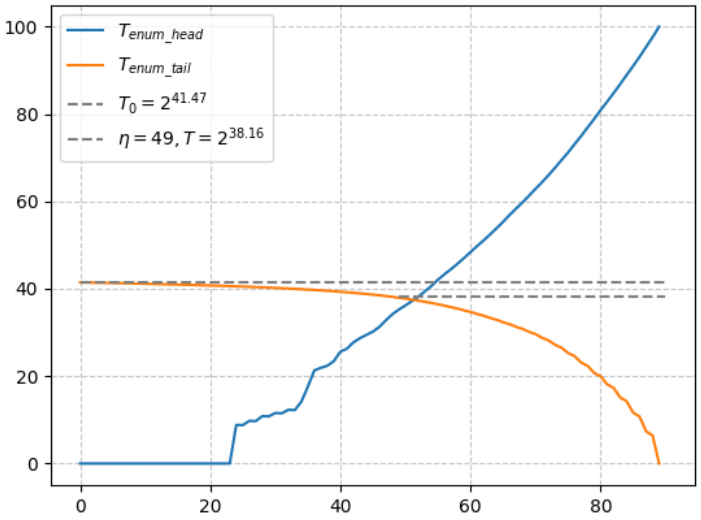



Fig. 12: Cost Evaluation for $N = 90$ and $p = 0.03$

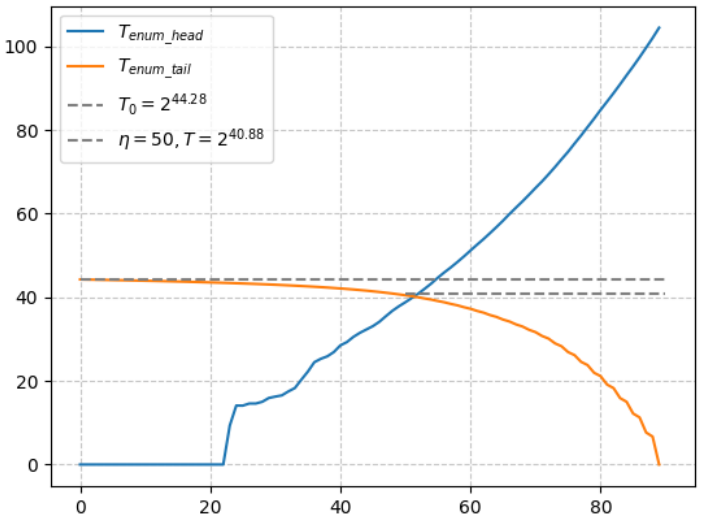



Fig. 13: Cost Evaluation for $N = 90$ and $p = 0.09$

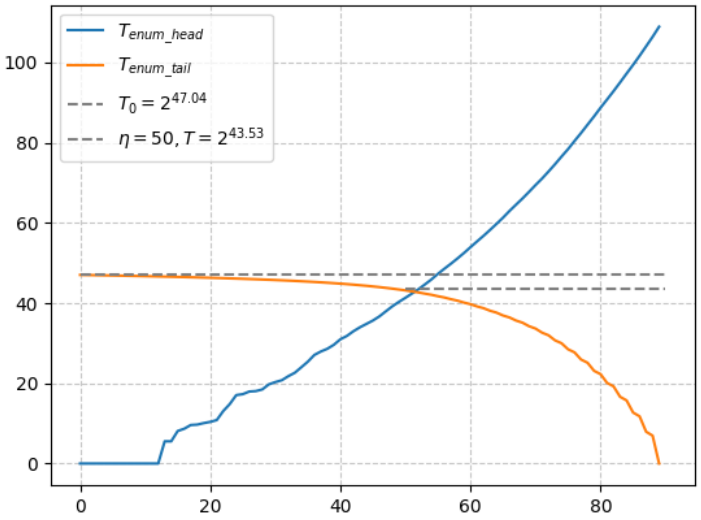



Fig. 14: Cost Evaluation for $N = 90$ and $p = 0.21$

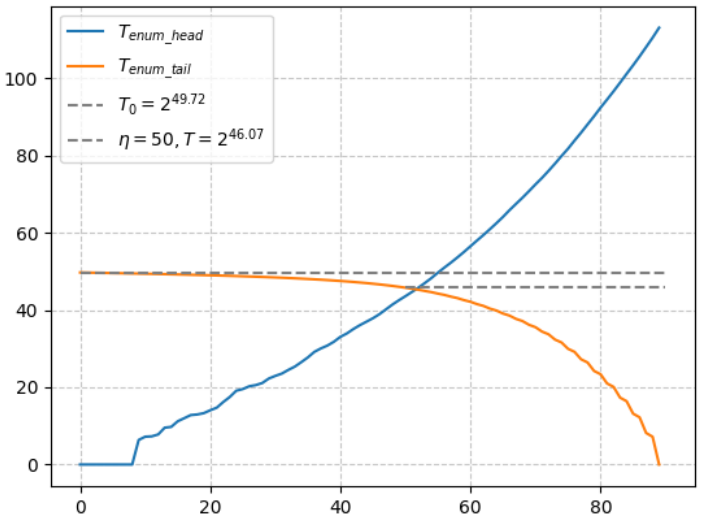



Fig. 15: Cost Evaluation for $N = 90$ and $p = 0.40$